Course : Database Management System

Code : BCS403

Semester : IV

Notes : MODULE-I

CANARA ENGINEERING COLLEGE

TOPICS

Introduction to Databases

Introduction, Characteristics of database approach

Advantages of using the DBMS approach

History of database applications.

Overview of Database Languages and Architectures

Data Models, Schemas, and Instances.

Three schema architecture and data independence.

Database languages and interfaces.

The Database System environment.

Conceptual Data Modelling using Entities and Relationships

Entity types, Entity sets, attributes, roles, and structural constraints.

Weak entity types.

ER diagrams.

ER diagram Examples.

Specialization and Generalization.

Question Bank

CANARA ENGINEERING COLLEGE

Session-1: Introduction To Databases: Introduction, Characteristics Of Database Approach, Advantages of Using the DBMS Approach

INTRODUCTION

- A **database** is a collection of related data. Here **data** means known facts that can be recorded and that have implicit meaning.
- **For example**, consider the names, telephone numbers, and addresses of the people we know. We may have recorded this data in an indexed address book or you may have stored it on a hard drive, using a personal computer and software such as Microsoft Access or Excel. This collection of related data with an implicit meaning is a database.
- **A database has the following implicit properties**:
  - A database represents some aspect of the real world, sometimes called the **miniworld** or the **universe of discourse (UoD)**. Changes to the miniworld are reflected in the database.
  - A database is a logically coherent collection of data with some inherent meaning. A random assortment of data cannot correctly be referred to as a database.
  - A database is designed, built, and populated with data for a specific purpose. It has an intended group of users and some preconceived applications in which these users are interested.
- A **database management system (DBMS)** is a collection of programs that enables users to create and maintain a database. **The DBMS is a general-purpose software system that facilitates the processes of defining, constructing, manipulating, and sharing databases among various users and applications**.
  - **Defining** a database involves specifying the data types, structures, and constraints of the data to be stored in the database. The database definition or descriptive information is also stored by the DBMS in the form of a database catalog or dictionary, it is called **meta-data**. **Constructing** the database is the process of storing the data on some storage medium that is controlled by the DBMS.

- **Manipulating** a database includes functions such as querying the database to retrieve specific data, updating the database to reflect changes in the miniworld, and generating reports from the data.
- **Sharing** a database allows multiple users and programs to access the database simultaneously.

- An **application program** accesses the database by sending queries or requests for data to the DBMS. A **query** typically causes some data to be retrieved. A **transaction** may cause some data to be read and some data to be written into the database.
- Other important functions provided by the DBMS include **protecting** the database and **maintaining** it over a long period of time.

The below figure illustrates the database system. The database and DBMS software together is called as a **database system**.

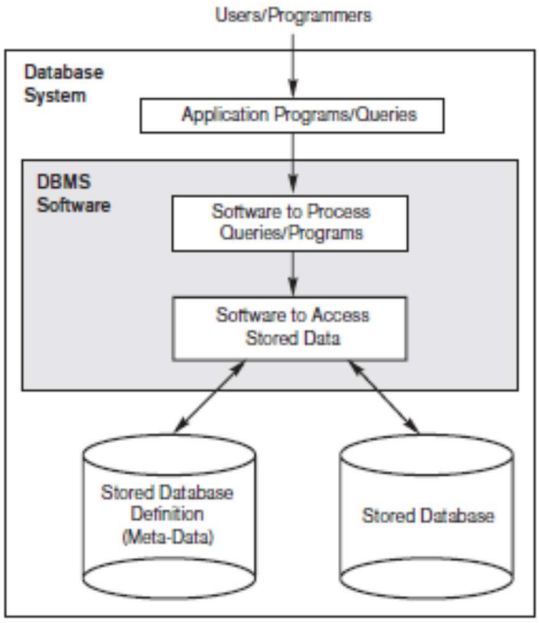


Figure 1.1
A simplified database system environment.

AN EXAMPLE DATABASE

- Consider an UNIVERSITY database for maintaining information concerning students, courses, and grades in a university environment. The below Figure 1.2 shows the database structure and a few sample data for such a database.
- The database is organized as five files, each of which stores **data records** of the same type.

- The STUDENT file stores data on each student, the COURSE file stores data on each course, the SECTION file stores data on each section of a course, the GRADE_REPORT file stores the grades that students receive in the various sections they have completed, and the PREREQUISITE file stores the prerequisites of each course.
- To **define** this database, we must specify the structure of the records of each file by specifying the different types of **data elements** to be stored in each record.

  For example: each STUDENT record includes data to represent the student's Name, Student number, Class and Major(like CS)such as freshman or '1', sophomore or '2', and so forth), and records and have *many relationships* among the records.
- We must also specify a **data type** for each data element within a record.

  For example, we can specify that Name of STUDENT is a string of alphabetic characters, Student number of STUDENT is an integer, and Grade of GRADE_REPORT is a single character from the set {'A', 'B', 'C', 'D', 'F', 'I'}.
- **To construct** the UNIVERSITY database, we store data to represent each student, course, section, grade report, and prerequisite as a record in the appropriate file.
- The records in the various files may be related. For example, the record for Smith in the STUDENT file is related to two records in the GRADE_REPORT file that specify Smith's grades in two sections.
- Most medium-size and large databases include many types of records and have *many relationships* among the records.

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure-1.2:Adatabase that store student and course information

- Database **manipulation** involves querying and updating.

  Examples of queries are as follows:

  - A list of all courses and grades of 'Smith' may be retrieved.
  - List the prerequisites of the 'Database' course.

  Examples of updates include the following:

  - Change the class of 'Smith' to sophomore.
  - Create a new section for the 'Database' course for this semester.

DIFFERENCE BETWEEN FILE ORIENTED APPROACH & DATABASE APPROACH:

FILE ORIENTED APPROACH

- Data files are created and processed by programs written by programmers or users of files.

- File created by different users of the organization often contain redundant data.
- The application programs depend upon the structural properties of the files.
- The application programs Interact directly with the operating system.

Disadvantages of file oriented approach:

- Files are created and processed by special programs: it is not possible by a program to access freely the other files, but it is necessary in the real world: **Example:** if the manager in an organization has to promote an employee, he must be able to access different files like EMPLOYEE_PERFORMANCE file,EMPLOYEE_PROJ file and so on.
- Files developed by different employees in an organization may have redundant data: Uncontrolled redundancy in files increases the cost of the storage and complicates the problem of updates and effects the consistent states of the data.
- Application programs depend upon the structure of the data files they process: Any changes made in the data files structure will require changes to be made in the application programs which are time consuming and expensive.
- Excessive programming effort is required: Application programs directly interact with the operating system. Hence programmers have to do much more in terms of programming to implement DBMS functions. It leads to poor data control.

DATABASE APPROACH

The database approach overcomes the drawbacks of file-oriented systems by providing the following features:

- It allows files to be integrated to avoid data redundancies.
- The user can access any file as long as he does not violate security constraints.
- Multiple users can have access to the collection of files.
- Each user may access a part of the database (a view is a subset of data base).
- Database system support centralized data control.
- The security, consistency and accuracy are maintained by DBMS rather than the user programs.

CHARACTERISTICS OF THE DATABASE APPROACH

The main characteristics of database approach are as follows:

- **Self-describing nature of a database system.**
- **Insulation between programs and data, and data abstraction.**
- **Support of multiple views of the data.**
- **Sharing of data and multiuser transaction processing.**

Self-describing nature of a database system:

- The database system contains not only the database itself but also a complete definition or description of the database structure and constraints.
- The definition is stored in the DBMS catalog, which contains information such as the structure of each file, the type and storage format of each data item, and various constraints on the data. The information stored in the catalog is called **meta-data**, and it describes the structure of the primary database.

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....	....	.....
....	....	.....
....	....	.....
Prerequisite_number	XXXXNNNN	PREREQUISITE

Figure 1.3
An example of a database catalog for the database in Figure 1.2.

- A general-purpose DBMS software package is not written for a specific database application. Therefore, it must refer to the catalog to know the structure of the files in a specific database, such as the type and format of data it will access. The DBMS software must work equally well with any number of database applications.

Insulation between programs and data, and data abstraction:

- In file oriented approach, any changes to the structure of a file requires changing all programs that access that file. In DBMS access programs do not require such changes as the structure of data files is stored in the DBMS catalog separately from the access programs. This property is called **program-data independence**.
- **For example**, in a DBMS environment if we want to add another piece of data to each STUDENT record, say the Birth_date, we only need to change the description of STUDENT records in the catalog to reflect the inclusion of the new data item Birth_date, no programs are changed.
- In some types of database systems, such as object-oriented and object-relational systems users can define operations on data as part of the database definitions. User application programs can operate on the data by invoking these operations through their names and arguments, regardless of how the operations are implemented. This property is termed **program-operation independence**.
- The characteristic that allows program-data independence and program-operation independence is called **data abstraction**.

Support of Multiple Views of the Data:

- A database typically has many users, each of whom may require a different perspective or **view** of the database.
- A view may be a subset of the database or it may contain **virtual data** that is derived from the database files but is not explicitly stored.
- A multiuser DBMS whose users have a variety of distinct applications must provide facilities for defining multiple views.

Sharing of Data and Multiuser Transaction Processing:

- A multiuser DBMS must allow multiple users to access the database at the same time.
- The DBMS must include **concurrency control** software to ensure that several users trying to update the same data do so in a controlled manner so that the result of the updates is correct.
- For example, when several reservation agents try to assign a seat on an airline flight, the DBMS should ensure that each seat can be accessed by only one agent at a time for assignment to a passenger. These types of applications are generally called **online transaction processing (OLTP)** applications.
- A fundamental role of multiuser DBMS software is to ensure that concurrent transactions operate correctly and efficiently.
- A **transaction** is an executing program or process that includes one or more database accesses, such as reading or updating of database records.
- The **isolation** property ensures that each transaction appears to execute in isolation from other transactions.
- The **atomicity** property ensures that either all the database operations in a transaction are executed or none are.

ACTORS ON THE SCENE

The people whose jobs involve the day-to-day use of a large database are called as actors on the scene and are as follows:

- **Database Administrators**
- **Database Designers**
- **End Users**
- **System Analysts and Application Programmers (Software Engineers)**

Database Administrators

- In a database environment, the primary resource is the database itself, and the secondary resource is the DBMS and related software. Administering these resources is the responsibility of the **database administrator (DBA)**.

Database Designers

- Database designers are responsible for identifying the data to be stored in the database and for choosing appropriate structures to represent and store this data.

End Users

- End users are the people whose jobs require access to the database for querying, updating, and generating reports; the database primarily exists for their use.
- The following are the categories of end users:
  - **Casual end users:** They occasionally access the database, but they may need different information each time. They use a sophisticated database query language to specify their requests and are typically middle or high-level managers or other occasional browsers. Casual users learn only a few facilities of DBMS that they may use repeatedly.
  - **Naive or parametric end users:** They constantly query and update the database, using standard types of queries and updates called canned transactions that have been carefully programmed and tested. Naive end users need to learn very little about the facilities provided by the DBMS. Example: Reservation agents for airlines, hotels, and car rental companies check availability for a given request and make reservations.
  - **Sophisticated end users:** These include engineers, scientists, business analysts, and others who thoroughly familiarize themselves with the facilities of the DBMS in order to implement their own applications to meet their complex requirements.
  - **Standalone users:** They maintain personal databases by using ready-made program packages that provide easy-to-use menu-based or graphics-based interfaces. For example the user of a tax package that stores a variety of personal financial data for tax purposes.

System Analysts and Application Programmers (Software Engineers):

- **System analysts** determine the requirements of end users, especially naive and parametric end users, and develop specifications for standard canned transactions that meet these requirements.
- **Application programmers** implement these specifications as programs, then they test, debug, document, and maintain these canned transactions. Such analysts and programmers are commonly referred to as **software developers** or **software engineers.**

WORKERS BEHIND THE SCENE

The people who are associated with the design, development, and operation of the DBMS software and system environment are called as the workers behind the scene.

The following are categories of workers behind the scene:

- **DBMS system designers and implementers.**
- **Tool developers.**
- **Operators and maintenance personnel.**

DBMS system designers and implementers

They design and implement the DBMS modules and interfaces as a software package.

Tool developers

They design and implement **tools** which are the software packages that facilitate database modeling and design, database system design, and improved performance.

Operators and maintenance personnel

They are responsible for the actual running and maintenance of the hardware and software environment for the database system.

ADVANTAGES OF USING THE DBMS APPROACH

The following are the advantages of using a DBMS:

- Controlling Redundancy.
- Restricting Unauthorized Access.
- Providing Persistent Storage for Program Objects.
- Providing Storage Structures and Search Techniques for Efficient Query Processing.
- Providing Backup and Recovery.
- Providing Multiple User Interfaces.
- Representing Complex Relationships among Data.
- Enforcing Integrity Constraints.
- Permitting Inferencing and Actions Using Rules.

Controlling Redundancy:

- Storing the same data multiple times is called as **redundancy**
- **Redundancy** leads to several problems like:
  - Duplication of efforts: A single logical update, such as entering data on a new student need to be made multiple times, once for each file where student data is recorded.
  - Storage space is wasted: When the same data is stored repeatedly Storage space is wasted and this problem may be serious for large databases.
  - Inconsistency: The file representing the same data may become inconsistent because an update is applied to some of the files but not to others.
- In the database approach, the views of different user groups are integrated during database design. we can have a database design that stores each logical data item such as a student's name or birth date in only one place in the database. This is known as **data normalization**, and it ensures consistency and saves storage space.
- In some cases it is sometimes necessary to use **controlled redundancy** to improve the performance of queries. By placing all the data together, we do not have to search multiple files to collect this data. This is known as **denormalization**.

Restricting Unauthorized Access:

- When multiple users share a large database, some users will not be authorized to access all information in the database. For example, financial data is often considered confidential, and only authorized persons are allowed to access such data.
- Some users may only be permitted to retrieve data, whereas others are allowed to retrieve and update. Hence, users or user groups are given account numbers protected by passwords, which they can use to gain access to the database.
- A DBMS should provide a **security and authorization subsystem**, which the DBA uses to create accounts and to specify account restrictions. Then, the DBMS should enforce these restrictions automatically. For example, only the dba's staff may be allowed to use certain **privileged software**, such as the software for creating new accounts.
- Parametric users may be allowed to access the database only through the predefined canned transactions developed for their use.

Providing Persistent Storage for Program Objects:

- Databases can be used to provide **persistent storage** for program objects and data structures. This is one of the main reasons for **object-oriented database systems**. Object-oriented database systems are compatible with programming languages such as C++ and Java, and the DBMS software automatically performs any necessary conversions. Hence, a complex object in C++ can be stored permanently in an object-oriented DBMS. Such an object is said to be **persistent**, since it survives the termination of program execution and can later be directly retrieved by another C++ program.
- The persistent storage of program objects and data structures is an important function of database systems. Traditional database systems often suffered from the so called **impedance mismatch problem**, since the data structures provided by the DBMS were incompatible with the programming language's data structures. Object-oriented database systems typically offer data structure **compatibility** with one or more object-oriented programming languages.

Providing Storage Structures and Search Techniques for Efficient Query Processing:

- Database systems must provide capabilities for efficiently executing queries and updates. Because the database is typically stored on disk, the DBMS must provide specialized data structures and search techniques to speed up disk search for the desired records. Auxiliary files called **indexes** are used for this purpose.
- The DBMS often has a **buffering** or **caching** module that maintains parts of the database in main memory buffers to speed up retrieval of data through querying.
- The **query processing and optimization** module of the DBMS is responsible for choosing an efficient query execution plan for each query based on the existing storage structures.

Providing Backup and Recovery:

- A DBMS must provide facilities for recovering from hardware or software failures. The **backup and recovery subsystem** of the DBMS is responsible for recovery.
- For example, if the computer system fails in the middle of a complex update transaction, the recovery subsystem is responsible for making sure that the database is restored to the state it was in before the transaction started executing.

Providing Multiple User Interfaces:

- Many types of users with varying levels of technical knowledge use a database, a DBMS should provide a variety of user interfaces.
- Example:
  - query languages for casual users,
  - programming language interfaces for application programmers,
  - forms and command codes for parametric users,
  - menu-driven interfaces and natural language interfaces for standalone users.
- Both forms-style interfaces and menu-driven interfaces are commonly known as **graphical user interfaces (GUIs)**.

Representing Complex Relationships among Data:

A database may include numerous varieties of data that are interrelated in many ways. For example in university database, the record of a student in the STUDENT file is related to records in the GRADE_REPORT file.

Enforcing Integrity Constraints:

- Most database applications have certain **integrity constraints** that must hold for the data. A DBMS should provide capabilities for defining and enforcing these constraints. The simplest type of integrity constraint involves specifying a data type for each data item.
- For example: In university database we can specify that the value of the Class data item Within each STUDENT record must be a one digit integer and that the value of Name must be a string of no more than 30 alphabetic characters.

Permitting Inferencing and Actions Using Rules:

- Some database systems provide capabilities for defining deduction rules for inferencing new information from the stored database facts. Such systems are called **deductive database systems**.
- For example, there may be complex rules in the miniworld application for determining when a student is on probation. These can be specified *declaratively* as **rules**, which when compiled and maintained by the DBMS can determine all students on probation. In a traditional DBMS, an explicit
- More powerful functionality is provided by **active database systems**, which provide active rules that can automatically initiate actions when certain events and conditions occur.

ADDITIONAL IMPLICATIONS OF USING THE DATABASE APPROACH

The following are some additional implications of using the database approach that can benefit most organizations:

- **Potential for Enforcing Standards:**
  The database approach permits the DBA to define and enforce standards among database users in a large organization. This facilitates communication and cooperation among various departments, projects, and users within the organization

- **Reduced Application Development Time:**
  - A prime selling feature of the database approach is that developing a new application such as the retrieval of certain data from the database for printing a new report takes very little time.
  - Designing and implementing a large multiuser database from scratch may take more time than writing a single specialized file application.
  - Once a database is up and running, substantially less time is generally required to create new applications using DBMS facilities.
  - Development time using a DBMS is estimated to be one-sixth to one-fourth of that for a traditional file system.
- **Flexibility:**
  - It may be necessary to change the structure of a database as requirements change.
  - For example, a new user group may emerge that needs information not currently in the database.
  - Modern DBMSs allow certain types of evolutionary changes to the structure of the database without affecting the stored data and the existing application programs.
- **Availability of Up-to-Date Information:**

  A DBMS makes the database available to all users. As soon as one user's update is applied to the database, all other users can immediately see this update.
- **Economies of Scale:**
  - The DBMS approach permits consolidation of data and applications, thus reducing the amount of wasteful overlap between activities of data-processing personnel in different projects or departments as well as redundancies among applications.
  - This enables the whole organization to invest in more powerful processors, storage devices, or communication gear, rather than having each department purchase its own (lower performance) equipment. This reduces overall costs of operation and management.

1. What are the main characteristics of database approach over the file processing approach? (08marks)
2. What are the responsibilities of DBA and database designers? (04marks)
3. List the advantages and disadvantages of DBMS? Discuss any five advantages by comparing with file system? (08marks)
4. Define the terms
    a. Data
    b. Database
    c. Metadata
    d. Data Model
    e. Canned transactions
    f. Database designers (10 marks)
5. Discuss the main characteristics of database approach. How does it differ from the traditional file system? (10 marks)
6. What is meant by “persistent storage for program objects” explain. (05 marks)
7. Explain the advantages of DBMS. (10 marks)
8. List the different types of end users. (4 marks)

Session 2: A Brief History of Database Applications and Data Models

- **Early Database Applications Using Hierarchical and Network Systems:**
  - Many early database applications maintained records in large organizations such as corporations, universities, hospitals, and banks. In many of these applications, there were large numbers of records of similar structure. There were also many types of records and many interrelationships among them.
  - One of the main problems with early database systems was the intermixing of conceptual relationships with the physical storage and placement of records on disk.
  - These systems did not provide sufficient *data abstraction* and *program-data independence* capabilities. For example, the grade records of a particular student could be physically stored next to the student record.
  - Another shortcoming of early systems was that they provided only programming language interfaces. This made it time-consuming and expensive to implement new queries and transactions, since new programs had to be written, tested, and debugged.

- **Providing Data Abstraction and Application Flexibility with Relational Databases:**
  - Relational databases were originally proposed to separate the physical storage of data from its conceptual representation and to provide a mathematical foundation for data representation and querying.
  - The relational data model also introduced high-level query languages that provided an alternative to programming language interfaces, making it much faster to write new queries.
  - Relational systems provided flexibility to develop new queries quickly and to reorganize the database as requirements changed. Hence, *data abstraction* and *program-data independence* were much improved when compared to earlier systems.

- **Object-Oriented Applications and the Need for More Complex Databases:**
  - The emergence of object-oriented programming languages in the 1980s and the need to store and share complex, structured objects led to the development of object-oriented databases (OODBs).
  - They also incorporated many of the useful object-oriented paradigms, such as abstract data types, encapsulation of operations, inheritance, and object identity.
  - They are now mainly used in specialized applications, such as engineering design, multimedia publishing, and manufacturing systems.

- **Interchanging Data on the Web for E-Commerce Using XML:**
  - The World Wide Web provides a large network of interconnected computers. In the 1990s, electronic commerce (e-commerce) emerged as a major application on the Web.
  - The information on e-commerce Web pages were often dynamically extracted data from DBMSs. A variety of techniques were developed to allow the interchange of data on the Web.
  - Currently, eXtended Markup Language (XML) is considered to be the primary standard for interchanging data among various types of databases and Web pages. XML combines concepts from the models used in document systems with database modeling concepts.

- **Extending Database Capabilities for New Applications:**
  - The success of database systems in traditional applications encouraged developers of other types of applications to attempt to use them.
  - **Some examples** of these applications are **scientific applications**, Storage and retrieval of **images** and **videos, Data mining** applications **spatial** applications and **Time series** applications.

DATABASES VERSUS INFORMATION RETRIEVAL

- Traditionally, database technology applies to structured and formatted data that arises in routine applications in government, business, and industry.
- Database technology is heavily used in manufacturing, retail, banking, insurance, finance, and health care industries, where structured data is collected through forms, such as invoices or patient registration documents.
- An area related to database technology is **Information Retrieval (IR)**, which deals with books, manuscripts, and various forms of library-based articles. Data is indexed, cataloged, and annotated using keywords.
- IR is concerned with searching for material based on these keywords, and with the many problems dealing with document processing and free-form text processing.

WHEN NOT TO USE A DBMS

- The overhead costs of using a DBMS are due to the following:
- High initial investment in hardware, software, and training.
- The generality that a DBMS provides for defining and processing data.
- Overhead for providing security, concurrency control, recovery, and integrity functions.

It may be more desirable to use regular files under the following circumstances:

- Simple, well-defined database applications that are not expected to change at all
- Stringent, real-time requirements for some application programs that may not be met because of DBMS overhead
- Embedded systems with limited storage capacity, where a general-purpose DBMS would not fit
- No multiple-user access to data

DATA MODELS, SCHEMAS, AND INSTANCES

- **Data abstraction** refers to the suppression of details of data organization and storage, and the highlighting of the essential features for an improved understanding of data. One of the main characteristics of the database approach is to support data abstraction so that different users can perceive data at their preferred level of detail.
- A **data model** is a collection of concepts that can be used to describe the structure of a database. structure of a database includes the data types, relationships, and constraints that apply to the data. Most data models also include a set of **basic operations** for specifying retrievals and updates on the database.
- The concepts of **dynamic aspect** or **behavior** of a database application can also be included in the data model. This allows the database designer to specify a set of valid user-defined operations that are allowed on the database objects. An example of a user-defined operation could be COMPUTE_GPA, which can be applied to a STUDENT object.

CATEGORIES OF DATA MODELS

The data models are categorized according to the types of concepts they use to describe the database structure as follows:

- **High-level** or **conceptual data models**
- **low-level** or **physical data models**
- **Representational** or **implementation data models**

High-Level Or Conceptual Data Models:

- Conceptual data model provide concepts that are close to the way many users perceive data.
- Conceptual data models use concepts such as entities, attributes, and relationships.
- An **entity** represents a real-world object or concept, such as an employee or a project from the miniworld that is described in the database.
- An **attribute** represents some property of interest that further describes an entity, such as the employee's name or salary.

- A **relationship** among two or more entities represents an association among the entities, for example, a works-on relationship between an employee and a project.
- **Entity-Relationship model**—a popular high-level conceptual data model.

Low-Level Or **Physical Data Models:**

- Physical data model provide concepts that describe the details of how data is stored on the computer storage media, typically magnetic disks.
- Concepts provided by low-level data models are generally meant for computer specialists, not for end users.
- Physical data models describe how data is stored as files in the computer by representing information such as record formats, record orderings, and access paths.
- An **access path** is a structure that makes the search for particular database records efficient.
- An **index** is an example of an access path that allows direct access to data using an index term or a keyword.

Representational Or **Implementation Data Models:**

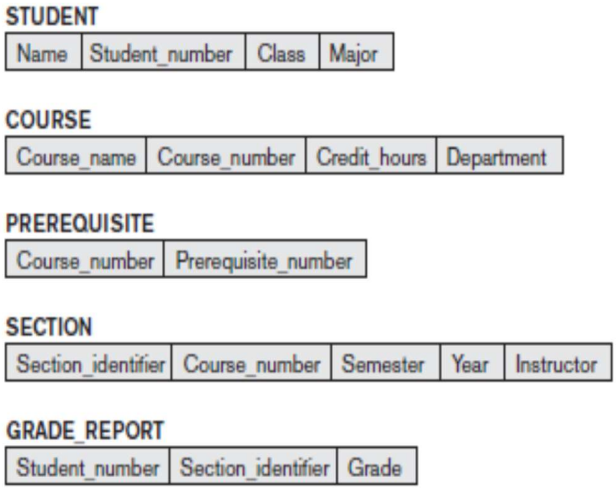
- Representational data model provide concepts that may be easily understood by end users but that are not too far removed from the way data is organized in computer storage. Representational data models hide many details of data storage on disk but can be implemented on a computer system directly.
- Representational or implementation data models are the models used most frequently in traditional commercial DBMSs. These include the widely used **relational data model**, as well as the so-called legacy data models the **network** and **hierarchical models.**
- Representational data models represent data by using record structures and hence are sometimes called **record-based data models**.
- The **object data model** is an example of a new family of higher-level implementation data models that are closer to conceptual data models.
- A standard for object databases called the ODMG object model has been proposed by the Object Data Management Group (ODMG).

Session 3: Schemas, And Instances. Three Schema Architecture and Data Independence

Schemas and Instances

- The description of a database is called the **database schema**, which is specified during database design and is not expected to change frequently.
- A displayed schema is called a **schema diagram**.

Figure 2.1
Schema diagram for the database in Figure 1.2.



- The Figure 2.1 shows a schema diagram for the University database. The diagram displays the structure of each record type but not the actual instances of records.
- The each object in the schema such as STUDENT or COURSE is called a **schema construct**.
- A schema diagram displays only *some aspects* of a schema, such as the names of record types and data items, and some types of constraints. Other aspects are not specified in the schema diagram such as the data type of each data item or the relationships among the various files.
- The data in the database at a particular moment in time is called a **database state** or **snapshot**. It is also called the *current* set of **occurrences** or **instances** in the database.
- In a given database state, each schema construct has its own *current set* of instances, for example, the STUDENT construct will contain the set of individual student entities (records) as its instances.
- Every time a record is inserted or deleted or the value of a data item in a record is changed, one state of the database is changed into another.

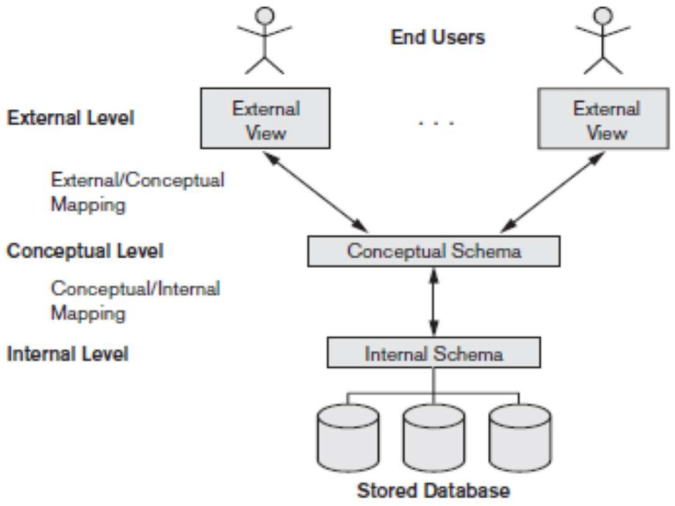
- When a new database is **defined**, its schema is specified only to the DBMS. At this point, the corresponding database state is the *empty state* with no data. The database gets its initial state when the database is first **populated** or **loaded** with the initial data. At any point in time, the database has a current state.
- The DBMS is partly responsible for ensuring that every state of the database is a **valid state,** that is, a state that satisfies the structure and constraints specified in the schema.
- The DBMS stores the descriptions of the schema constructs and constraints also called the **meta-data** in the DBMS catalog so that DBMS software can refer to the schema whenever it needs to.
- The schema is sometimes called the **intension**, and a database state is called an **extension** of the schema. Changing the database schema to represent the changes in the real world is known as **schema evolution**.

THREE-SCHEMA ARCHITECTURE AND DATA INDEPENDENCE

The Three-Schema Architecture

- The three schema architecture was proposed to help in achieving and visualizing the characteristics of the database approach. The below figure illustrates the Three Schema Architecture.

Figure 2.2
The three-schema architecture.



- The goal of the three-schema architecture is to separate the user applications from the physical database.
- In this architecture, schemas can be defined at the following three levels:
  - The **internal level** has an **internal schema**, which describes the physical storage structure of the database. The internal schema uses a physical data model and describes the complete details of data storage and access paths for the database.
  - The **conceptual level** has a **conceptual schema**, which describes the structure of the whole database for a community of users. The conceptual schema hides the details of physical storage structures and concentrates on describing entities, data types, relationships, user operations, and constraints. Usually, a representational data model is used to describe the conceptual schema when a database system is implemented. This *implementation conceptual schema* is often based on a *conceptual schema design* in a high-level data model.
  - The **external** or **view level** includes a number of **external schemas** or **user views**. Each external schema describes the part of the database that a particular user group is interested in and hides the rest of the database from that user group. As in the previous level, each external schema is typically implemented using a representational data model, possibly based on an external schema design in a high-level data model.
- The three-schema architecture is a convenient tool with which the user can visualize the schema levels in a database system. The processes of transforming requests and results between levels are called **mappings**.

Data Independence

- **Data independence** is the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.
- The following are the types of data independence:
  - **Logical data independence**
  - **Physical data independence**

- **Logical data independence**
  - It is the capacity to change the conceptual schema without having to change the external schemas or application programs.
  - Conceptual schema may be changed to expand the database, to change constraints, or to reduce the database by removing a record type or data item.
  - Only the view definition and the mappings need to be changed in a DBMS that supports logical data independence. After the conceptual schema undergoes a logical reorganization, application programs that reference the external schema constructs must work as before. Changes to constraints can be applied to the conceptual schema without affecting the external schemas or application programs.
  - Logical data independence is harder to achieve because it allows structural and constraint changes without affecting application programs.
- **Physical data independence**
  - It is the capacity to change the internal schema without having to change the conceptual schema. Hence, the external schemas need not be changed as well.
  - Changes to the internal schema may be needed because some physical files may be reorganized, for example, by creating additional access structures to improve the performance of retrieval or update.
  - If the same data as before remains in the database, there is no need to change the conceptual schema. For example, providing an access path to improve retrieval speed of section records in university database by semester and year should not require a query such as *list all sections offered in fall 2008* to be changed, although the query would be executed more efficiently by the DBMS by utilizing the new access path.
  - Physical data independence exists in most databases and file environments where physical details such as the exact location of data on disk, and hardware details of storage encoding, placement, compression, splitting, merging of records, and so on are hidden from the user. Applications remain unaware of these details.

- In a multiple-level DBMS, its catalog must be expanded to include information on how to map requests and data among the various levels. The DBMS uses additional software to accomplish these mappings by referring to the mapping information in the catalog.
- Data independence occurs because when the schema is changed at some level, the schema at the next higher level remains unchanged, only the *mapping* between the two levels is changed. Hence, application programs referring to the higher-level schema need not be changed.
- The three-schema architecture can make it easier to achieve true data independence, both physical and logical.

Review Questions

1. Explain different categories of data models? (06 marks)
2. Explain the three-schema architecture? (06 marks)
3. What is data independence? Bring out different types of independence. (06 marks)
4. List the differences between logical and physical data independence (05 marks)
5. Define the following and explain with an example each. (05 marks)
   a. Snapshot b) Intension c) Extension d) Schema construct
6. Define schema and instance. (04 marks)
7. What are the problems associated with three schema architecture. (02 marks)
8. Why would you choose a database system over flat files. When would make Sense not to use database system. (10 marks)

Session 4: Database Languages and Interfaces, the Database System Environment.

DBMS LANGUAGES

- Once the design of a database is completed and a DBMS is chosen to implement the database, the first step is to specify conceptual and internal schemas for the database.
- In many DBMSs where no strict separation of levels is maintained, one language, called the **data definition language** (**DDL**), is used by the DBA and by database designers to define both internal and conceptual schemas. The DBMS will have a DDL compiler whose function is to process DDL statements in order to identify descriptions of the schema constructs and to store the schema description in the DBMS catalog.
- In DBMSs where a clear separation is maintained between the conceptual and internal levels, the DDL is used to specify the conceptual schema only. Another language, the **storage definition language** (**SDL**), is used to specify the internal schema. The mappings between the two schemas may be specified in either one of these languages.
- For a true three-schema architecture, needs a third language, the **view definition language** (**VDL**), to specify user views and their mappings to the conceptual schema, but in most DBMSs *the DDL is used to define both conceptual and external schemas*.
- In relational DBMSs, SQL is used in the role of VDL to define user or application **views** as results of predefined queries.
- Once the database schemas are compiled and the database is populated with data, users must have some means to manipulate the database. Typical manipulations include retrieval, insertion, deletion, and modification of the data. The DBMS provides a set of operations or a language called the **data manipulation language** (**DML**) for these purposes.
- A typical example of a comprehensive database language is the SQL relational database language , which represents a combination of DDL, VDL, and DML, as well as statements for constraint specification, schema evolution, and other features.

- There are two main types of DMLs:
  - A **high-level** or **nonprocedural** DML can be used on its own to specify complex database operations concisely. Many DBMSs allow high-level DML statements either to be entered interactively from a display monitor or terminal or to be embedded in a general-purpose programming language. They are also called **set-at-a-time** or **set-oriented** DMLs. Example SQL. A high-level DML often specifies which data to retrieve rather than how to retrieve it therefore, such languages are also called **declarative**. A high-level DML used in a standalone interactive manner is called a **query language**.
  - A **low-level** or **procedural** DML must be embedded in a general-purpose programming language. This type of DML typically retrieves individual records or objects from the database and processes each separately. Therefore, it needs to use programming language constructs, such as looping, to retrieve and process each record from a set of records. Low-level DMLs are also called **record-at-a-time** DMLs because of this property. Example: DL/1
- A query in, whether high level or low level, are embedded in a general-purpose programming language, that language is called the **host language** and the DML is called the **data sublanguage**.
- Casual end users typically use a high-level query language to specify their requests, whereas programmers use the DML in its embedded form. For naive and parametric users, there usually are **user-friendly interfaces** for interacting with the database.

DBMS INTERFACES

User-friendly interfaces provided by a DBMS may include the following:

- **Menu-Based Interfaces for Web Clients or Browsing**
  - These interfaces present the user with lists of options (called **menus)** that lead the user through the formulation of a request. The query is composed step by step by picking options from a menu that is displayed by the system.

- Pull-down menus are a very popular technique in **Web-based user interfaces**. They are also often used in **browsing interfaces**, which allow a user to look through the contents of a database in an exploratory and unstructured manner.

- **Forms-Based Interfaces**
  - A forms-based interface displays a form to each user. Users can fill out all of the **form** entries to insert new data, or they can fill out only certain entries, in which case the DBMS will retrieve matching data for the remaining entries.
  - Forms are usually designed and programmed for naive users as interfaces to canned transactions. Many DBMSs have **forms specification languages**, which are special languages that help programmers, specify such forms.
- **Graphical User Interfaces**
  - A GUI typically displays a schema to the user in diagrammatic form. The user then can specify a query by manipulating the diagram.
  - Most GUIs use a **pointing device**, such as a mouse, to select certain parts of the displayed schema diagram.
- **Natural Language Interfaces**
  - These interfaces accept requests written in English or some other language and attempt to *understand* them. A natural language interface usually has its own *schema*.
  - The natural language interface refers to the words in its schema, as well as to the set of standard words in its dictionary, to interpret the request. If the interpretation is successful, the interface generates a high-level query corresponding to the natural language request and submits it to the DBMS for processing otherwise, a dialogue is started with the user to clarify the request.
- **Speech Input and Output**
  - Applications with limited vocabularies such as inquiries for telephone directory, flight arrival/departure, and credit card account information are allowing speech for input and output to enable customers to access this information.

- The speech input is detected using a library of predefined words and used to set up the parameters that are supplied to the queries. For output, a similar conversion from text or numbers into speech takes place.

- **Interfaces for Parametric Users**
  - Parametric users, such as bank tellers, often have a small set of operations that they must perform repeatedly. For example, a teller is able to use single function keys to invoke routine and repetitive transactions such as account deposits or withdrawals, or balance inquiries.
  - Systems analysts and programmers design and implement a special interface for each known class of naive users. Usually a small set of abbreviated commands is included, with the goal of minimizing the number of keystrokes required for each request. For example, function keys in a terminal can be programmed to initiate various commands. This allows the parametric user to proceed with a minimal number of keystrokes.
- **Interfaces for the DBA.**
  - Most database systems contain privileged commands that can be used only by the DBA staff.
  - These include commands for creating accounts, setting system parameters, granting account authorization, changing a schema, and reorganizing the storage structures of a database.

THE DATABASE SYSTEM ENVIRONMENT

DBMS COMPONENT MODULES

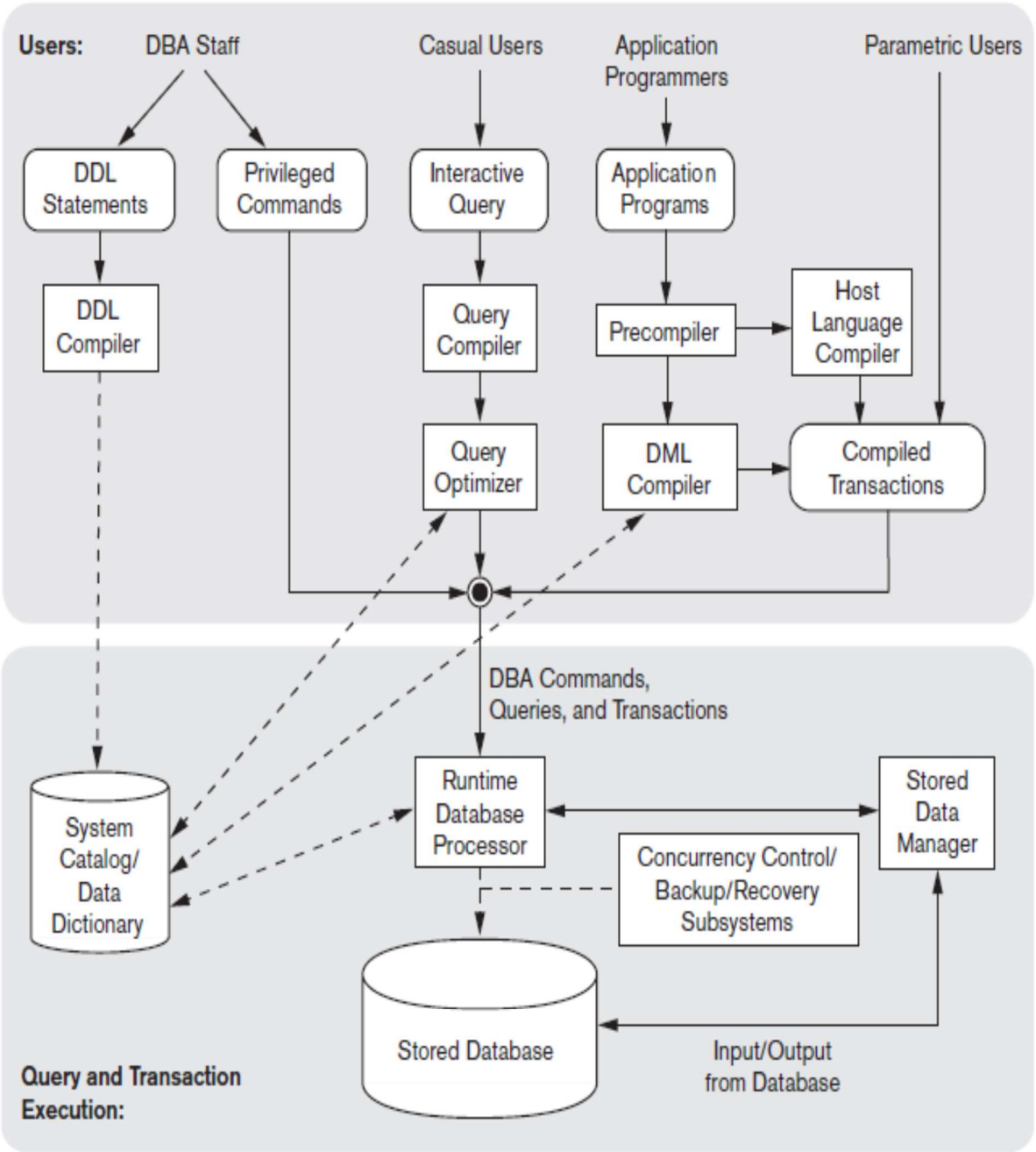


Figure 2.3
Component modules of a DBMS and their interactions.

- The Figure 2.3 above illustrates, in a simplified form, the typical DBMS components.
- The figure is divided into two parts:
  - The top part of the figure refers to the various users of the database environment and their interfaces.
  - The lower part shows the internals of the DBMS responsible for storage of data and processing of transactions.
- The database and the DBMS catalog are usually stored on disk. Access to the disk is controlled primarily by the **operating system** (**OS**), which schedules disk read/write.
- Many DBMSs have their own **buffer management** module to schedule disk read/write, because this has a considerable effect on performance.
- A higher-level **stored data manager** module of the DBMS controls access to DBMS information that is stored on disk, whether it is part of the database or the catalog.
- The top part of Figure 2.3 shows interfaces for the DBA staff, casual users application programmers and parametric users.
- The DBA staff works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands. The DDL compiler processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog. The catalog includes information such as the names and sizes of files, data types of data items and so on.
- Casual users and persons with occasional need for information from the database interact using some form of interface, which we call the **interactive query** interface. These queries are parsed and validated for correctness by a **query compiler** that compiles them into an internal form. This internal query is subjected to query optimization.
- The **query optimizer** is concerned with the rearrangement and possible reordering of operations, elimination of redundancies, and use of correct algorithms and indexes during execution. It consults the system catalog for statistical and other physical information about the stored data and generates executable code that performs the necessary operations for the query and makes calls on the runtime processor.

- Application programmers write programs in host languages such as Java, C, or C++ that are submitted to a precompiler. The **precompiler** extracts DML commands from an application program written in a host programming language. These commands are sent to the DML compiler for compilation into object code for database access. The rest of the program is sent to the host language compiler. The object codes for the DML commands and the rest of the program are linked, forming a canned transaction whose executable code includes calls to the runtime database processor.
- In the lower part of Figure 2.3, the **runtime database processor** executes the privileged commands, the executable query plans, and the canned transactions with runtime parameters. It works with the **system catalog** and may update it with statistics. It also works with the **stored data manager**, which in turn uses basic operating system services for carrying out low-level input/output (read/write) operations between the disk and main memory.
- The figure 2.3 shows **concurrency control** and **backup and recovery systems** as separate modules. They are integrated into the working of the runtime database processor for purposes of transaction management.

DATABASE SYSTEM UTILITIES

Most DBMSs have **database utilities** that help the DBA manage the database system.

Common utilities have the following types of functions:

- **Loading**

  A loading utility is used to load existing data files such as text files or sequential files into the database. Usually, the current (source) format of the data file and the desired (target) database file structure are specified to the utility, which then automatically reformats the data and stores it in the database. Some vendors are offering products that generate the appropriate loading programs, given the existing source and target database storage descriptions (internal schemas). Such tools are also called **conversion tools**. Example: Cincom's SUPRA Server SQL.

- **Backup**

  A backup utility creates a backup copy of the database, usually by dumping the entire database onto tape or other mass storage medium. The backup copy can be used to restore the database in case of catastrophic disk failure.

- **Database storage reorganization**

  This utility can be used to reorganize a set of database files into different file organizations, and create new access paths to improve performance.

- **Performance monitoring**

  Such a utility monitors database usage and provides statistics to the DBA. The DBA uses the statistics in making decisions such as whether or not to reorganize files or whether to add or drop indexes to improve performance.

TOOLS, APPLICATION ENVIRONMENTS AND COMMUNICATIONS FACILITIES

- Other tools are often available to database designers, users, and the DBMS.
- CASE tools12 are used in the design phase of database systems. Another tool that can be quite useful in large organizations is an expanded **data dictionary** (or **data repository**) **system**. In addition to storing catalog information about schemas and constraints, the data dictionary stores other information, such as design decisions, usage standards, application program descriptions, and user information. Such a system is also called an **information repository**.
- **Application development environments** such as PowerBuilder (Sybase) or JBuilder (Borland), have been quite popular. These systems provide an environment for developing database applications and include facilities that help in many facets of database systems, including database design, GUI development, querying and updating, and application program development.
- The DBMS also needs to interface with **communications software**, whose function is to allow users at locations remote from the database system site to access the database through computer terminals, workstations, or personal computers. The integrated DBMS and data communications system is called a **DB/DC** system.

CENTRALIZED AND CLIENT/SERVER ARCHITECTURES FOR DBMSS

CENTRALIZED DBMSs ARCHITECTURE

As prices of hardware declined, terminals were replaced with PCs and workstations but DBMS itself was still a **centralized** DBMS in which all the DBMS functionality, application program execution, and user interface processing were carried out on one machine. Figure 2.4 illustrates the physical components in a centralized architecture.

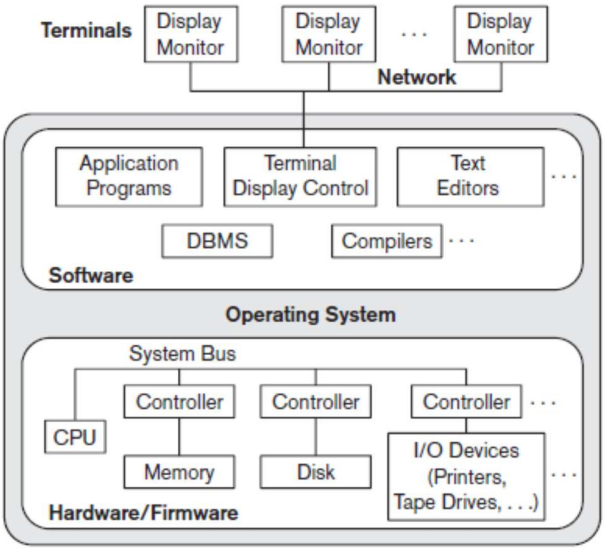


Figure 2.4
A physical centralized architecture.

BASIC CLIENT/SERVER ARCHITECTURES

The idea is to define **specialized servers** with specific functionalities. The resources provided by specialized servers can be accessed by many client machines.

- Examples

- **File server** maintains the files of the client machines which are to a number of PCs or small workstations connected to it.

- **Print server** handles all print requests by the client machines connected to it.

- **Web servers** or **e-mail servers** also fall into the specialized server category.

➢ A **client** is typically a user machine that provides user interface capabilities and local processing.

➢ A **server** is a system containing both hardware and software that can provide services to the client machines, such as file access, printing, archiving, or database access.

➢ In general, some machines install only client software, others only server software, and still others may include both client and server software as shown in the below figure 2.6 the physical two-tier client/server architecture.

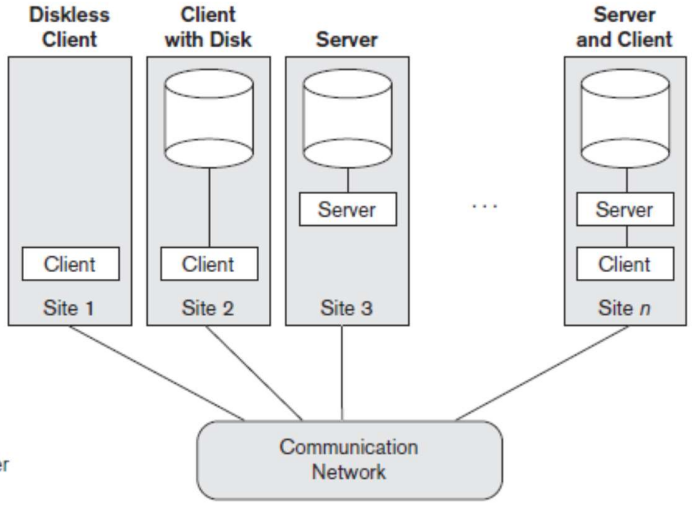


Figure 2.6
Physical two-tier client/server architecture.

TWO-TIER CLIENT/SERVER ARCHITECTURES FOR DBMSS

➢ In relational database management systems (RDBMSs), the query and transaction functionality related to SQL processing remained on the server side. In such architecture, the server is often called a **query server** or **transaction server** because it provides these two functionalities. In an RDBMS, the server is also often called an **SQL server**.

➢ The user interface programs and application programs can run on the client side. When DBMS access is required, the program establishes a connection to the DBMS which is on

the server side. Once the connection is created, the client program can communicate with the DBMS.

- The advantages of this architecture are its simplicity and seamless compatibility with existing systems.

THREE-TIER AND N-TIER ARCHITECTURES FOR WEB APPLICATIONS

- The emergence of the Web changed the roles of clients and servers, leading to the three-tier architecture.
- Many Web applications use the three-tier architecture, which adds an intermediate layer between the client and the database server, as illustrated in Figure 2.7(a).
- This intermediate layer or middle tier is called the application server or the Web server, depending on the application. This server plays an intermediary role by running application programs and storing business rules that are used to access data from the database server.

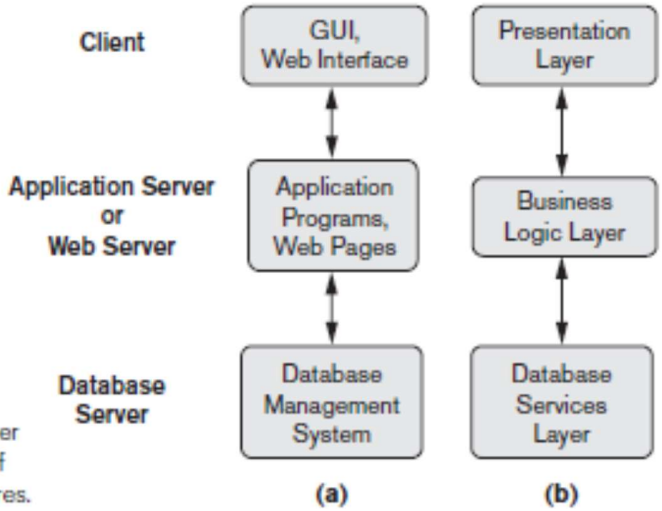


Figure 2.7
Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.

- Clients contain GUI interfaces and some additional application-specific business rules. The intermediate server accepts requests from the client, processes the request and sends database queries and commands to the database server, and then acts as a conduit for passing (partially) processed data from the database server to the clients, where it may be processed further and presented to users in GUI format. The user interface, application rules, and data access act as the three tiers.

- Figure 2.7(b) shows another architecture used by database and other application package vendors. The presentation layer displays information to the user and allows data entry. The business logic layer handles intermediate rules and constraints before data is passed up to the user or down to the DBMS.

CLASSIFICATION OF DATABASE MANAGEMENT SYSTEMS

The DBMS can be classified based on the data model as follows:

- Relational DBMS which are based on relational data model. Example: oracle
- Object Oriented DBMS which are based on Object data model.
- Object-Relational DBMS.
- Hierarchical DBMS which are based on hierarchical data model. Examples: IMS (IBM),System 2K (SAS Inc.) and TDMS.
- Network DBMS which are based on network data model and reasonable percentage of Worldwide-computerized data is still in these so-called legacy database systems. Examples: IDMS, DMS 1100 VAXDBMS and SUPRA.

The DBMS can be classified based on the number of users supported by the system as follows:

- Single-user systems support only one user at a time and are mostly used with PCs.
- Multiuser systems, which include the majority of DBMSs, support concurrent multiple users.

The DBMS can be classified based on the number of sites over which the database is distributed as follows:

- CENTRALIZED
  - A DBMS is Centralized if the data is stored at a single computer site.
  - A centralized DBMS can support multiple users, but the DBMS and the database reside totally at a single computer site.
- DISTRIBUTED
  - A Distributed DBMS (DDBMS) can have the actual database and DBMS software distributed over many sites, connected by a computer network.
  - Homogeneous DDBMSs use the same DBMS software at all the sites.

- Heterogeneous DDBMSs can use different DBMS software at each site.
- Federated DBMS or multidatabase system, in which the participating DBMSs are loosely coupled and have a degree of local autonomy.

The DBMS can be classified based on cost as follows:

- Open source or free DBMS products like MySQL and PostgreSQL that are supported by third-party vendors with additional services.
- Commercial DBMS like oracle or Standalone single user versions of some like Microsoft Access are sold per copy or included in the overall configuration of a desktop or laptop.

The DBMS can be classified based on the Purpose as follows

- General purpose DBMS
- Special purpose DBMS

Review Questions

1. Explain the typical components module of a DBMS with neat diagram. (10 marks)
2. With neat diagram explain the operation of two-tier client/server architecture for the RDBMS? (08 marks)
3. What are the data base utilities list few functions that utilities perform. (08 marks)
4. Discuss the criteria used to classify DBMS. (06 marks)
5. Define the terms: DDL, DML and DCL with examples. (06 marks)
6. Define and explain the importance of database catalog and explain the internal storage format of a catalog with example. (06 marks)
7. Discuss different types of user friendly interfaces and the types of users who typically use each? (08 marks)

Session 5: Conceptual Data Modelling Using Entities And Relationships: Entity Types, Entity Sets

USING HIGH-LEVEL CONCEPTUAL DATA MODELS FOR DATABASE DESIGN

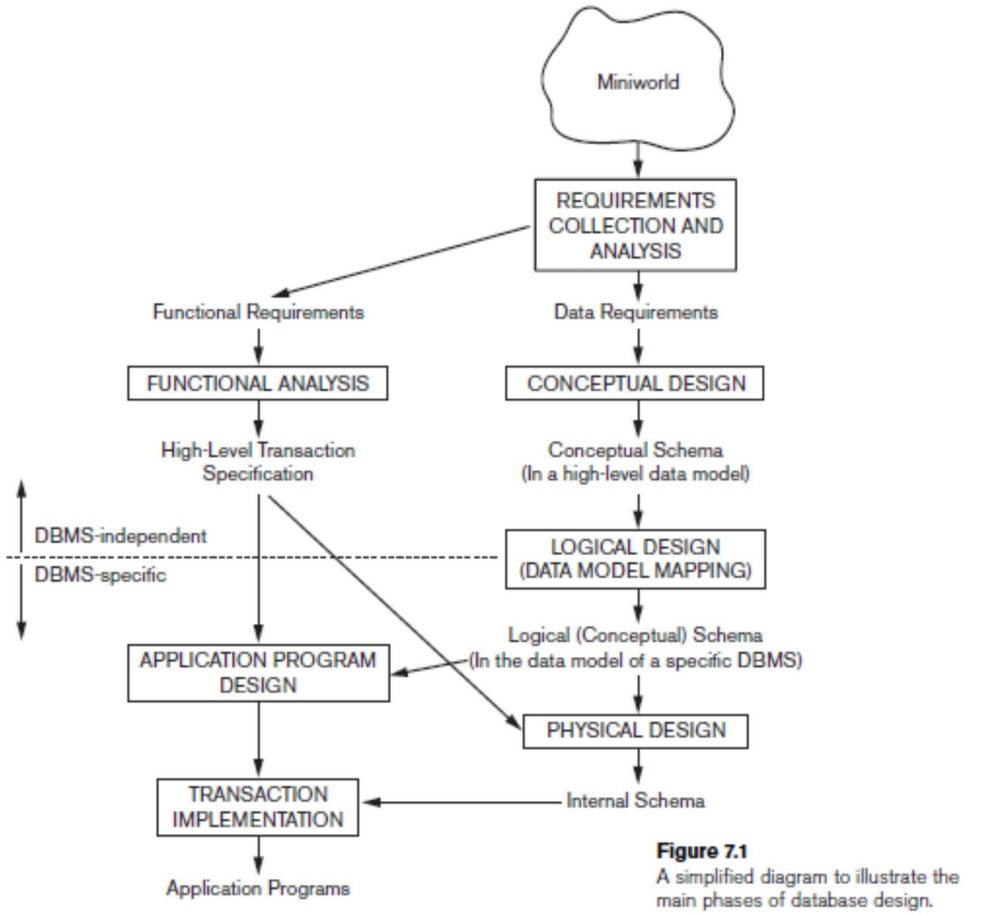


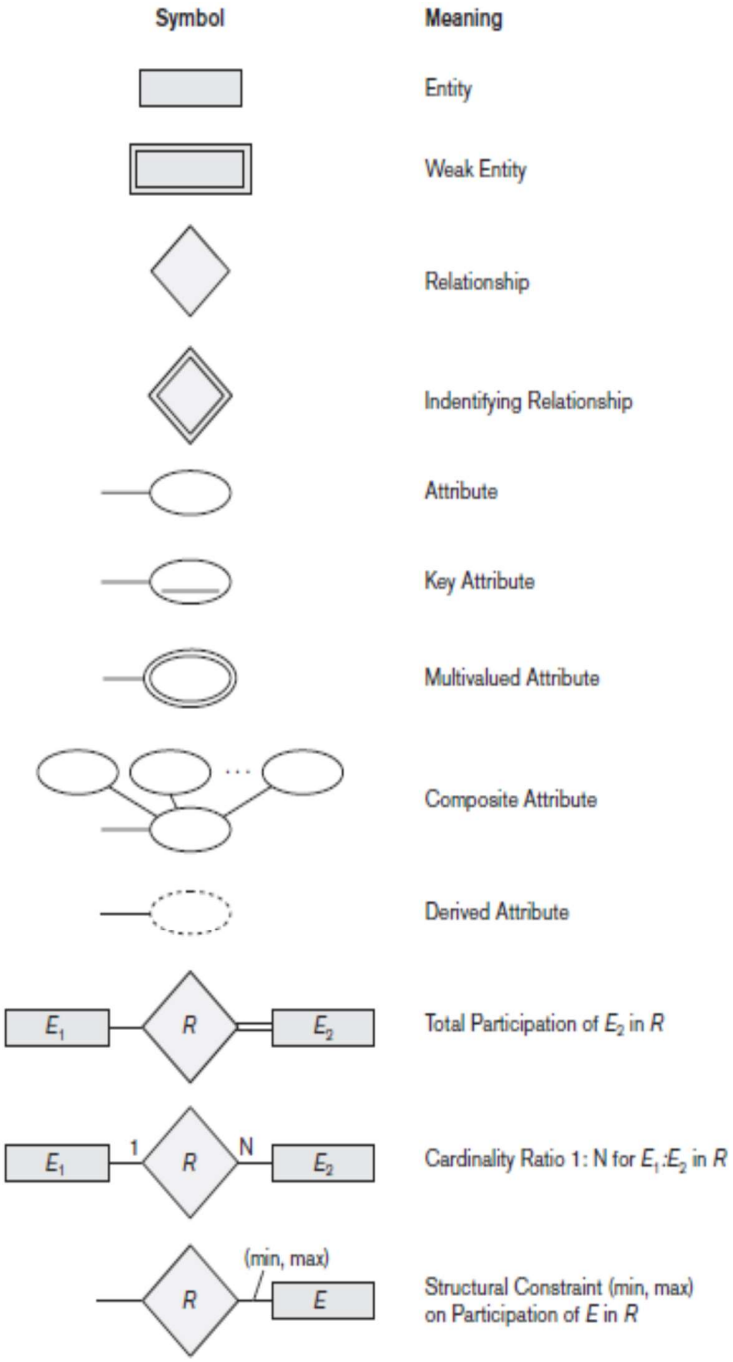
Figure 7.1
A simplified diagram to illustrate the main phases of database design.

- The above Figure 7.1 shows a simplified overview of the database design process. The first step shown is **requirements collection and analysis**. During this step, the database designers interview prospective database users to understand and document their **data requirements**. The result of this step is a concisely written set of users' requirements.

- In parallel with specifying the data requirements, it is useful to specify the known **functional requirements** of the application. These consist of the user defined **operations** (or **transactions**) that will be applied to the database, including both retrievals and updates.

- Once the requirements have been collected and analyzed, the next step is to create a **conceptual schema** for the database, using a high-level conceptual data model. This step is called **conceptual design**. The conceptual schema is a concise description of the data requirements of the users and includes detailed descriptions of the entity types, relationships, and constraints which are expressed using the concepts provided by the high-level data model.
- During or after the conceptual schema design, the basic data model operations can be used to specify the high-level user queries and operations identified during functional analysis.
- The next step in database design is the actual implementation of the database, using a commercial DBMS. Most current commercial DBMSs use an implementation data model, such as the relational or the object-relational database model to transform the conceptual schema d from the high-level data model into the implementation data model. This step is called **logical design** or **data model mapping**
- The last step is the **physical design** phase, during which the internal storage structures, file organizations, indexes, access paths, and physical design parameters for the database files are specified.
- In parallel with these activities, application programs are designed and implemented as database transactions corresponding to the highlevel transaction specifications.

Entity-Relationship(ER) Diagram Notations



A SAMPLE DATABASE APPLICATION

Consider a sample database application, called COMPANY, which serves to illustrate the basic ER model concepts and their use in schema design. The COMPANY database keeps track of a company's employees, departments, and projects.

- Suppose that after the requirements collection and analysis phase, the database designers provide the following description of the miniworld that is the part of the company that will be represented in the database.
- The company is organized into departments. Each department has a unique name, a unique number, and a particular employee who manages the department. We keep track of the start date when that employee began managing the department. A department may have several locations.
- A department controls a number of projects, each of which has a unique name, a unique number, and a single location.
- We store each employee's name, Social Security number, address, salary, sex (gender), and birth date. An employee is assigned to one department, but may work on several projects, which are not necessarily controlled by the same department. We keep track of the current number of hours per week that an employee works on each project. We also keep track of the direct supervisor of each employee (who is another employee).
- We want to keep track of the dependents of each employee for insurance purposes. We keep each dependent's first name, sex, birth date, and relationship to the employee.

The below figure 7.2 shows ER- diagram for company schema with above mentioned assumptions.

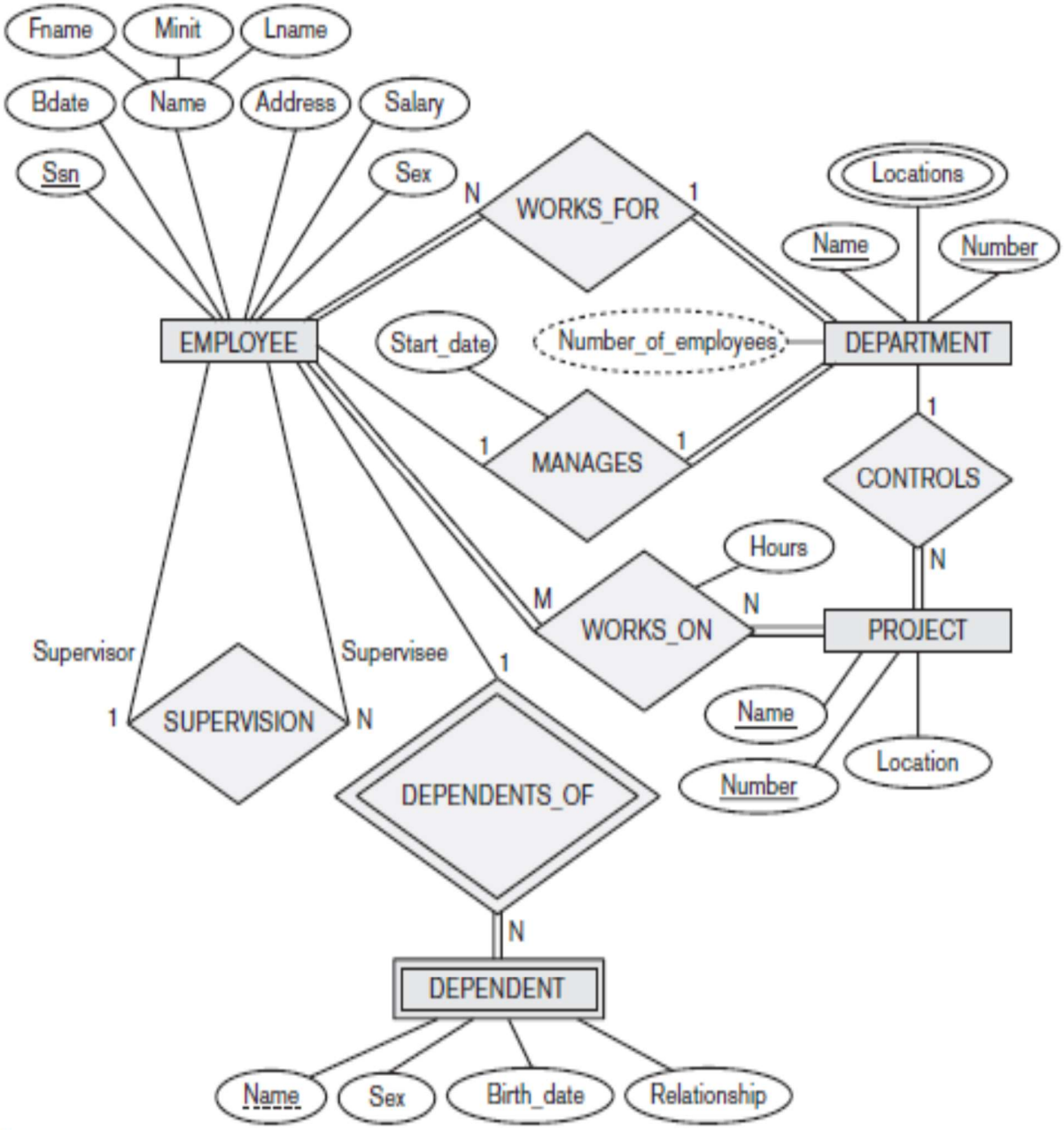


Figure 7.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 7.14.

Session 6: ENTITY TYPES, ENTITY SETS, ATTRIBUTES, AND KEYS

- The ER model describes data as entities, relationships, and attributes.

- **ENTITIES**
  - The basic object that the ER model represents is an **entity,** which is a **thing** in the real world with **an independent existence**.
  - An entity may be an object with a physical existence, such as an employee or it may be an object with a conceptual existence such as a company.
  - Examples: student, car, company, university, etc.
- **ATTRIBUTES**
  - Each entity has **attributes**, which are the **particular properties** that describe it.
  - **For example**, an EMPLOYEE entity may be described by the employee's name, age, address, salary, and job.
  - A particular entity will have a **value** for each of its attributes.
  - The below Figure 7.3 shows two entities and the values of their attributes. The EMPLOYEE entity *e*1 has four attributes: Name, Address, Age, and Home_phone and their values are 'John Smith,' '2311 Kirby, Houston, Texas 77001', '55', and '713-749-2630', respectively.
  - The COMPANY entity *c*1 has three attributes: Name, Headquarters, and President and their values are 'Sunco Oil', 'Houston', and 'John Smith', respectively.

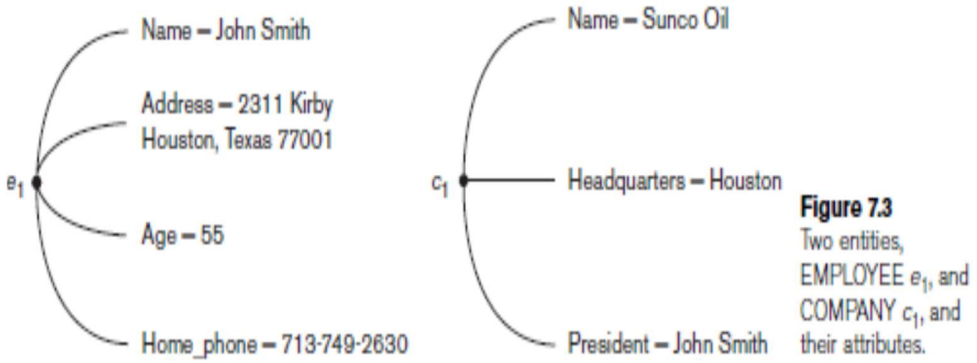


Figure 7.3
Two entities, EMPLOYEE e_1, and COMPANY c_1, and their attributes.

- **Types of attributes**
  - simple attributes.

- Composite attributes.
- Singlevalued attributes.
- Multivalued attributes.
- stored and derived attributes.

- Simple attributes:
  - Attributes that are not divisible are called **simple** or **atomic attributes**.
  - Example: SSN and gender attribute of an employee entity.
- Composite attributes:
  - Attributes that can be divided into smaller subparts, which represent more basic attributes with independent meanings are called as **Composite attributes**.
  - For Example, the Address attribute of the EMPLOYEE entity can be subdivided into Street_address, City, State, and Zip.
  - Composite attributes can form a hierarchy like Number, Street, and Apartment_number, as shown in below Figure 7.4.
  - Composite attributes are useful to model situations in which a user sometimes refers to the composite attribute as a unit but at other times refers specifically to its components.

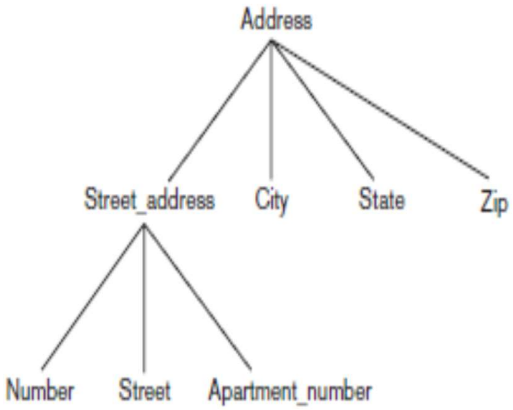


Figure 7.4
A hierarchy of composite attributes.

- Single-Valued attributes:

- The attributes that have a single value for a particular entity are called as **single-valued attributes**.
- Example: Age and SSN are single-valued attributes of an employee.

- Multivalued attributes:
  - The attribute that can have a set of values for the same entity is called **multivalued attribute**.
  - Examples: Colors attribute for a car or a College_degrees attribute for a person.
  - A multivalued attribute may have lower and upper bounds to constrain the *number of values* allowed for each individual entity.
  - For example, the Colors attribute of a car may be restricted to have between one and three values, if we assume that a car can have three colors at most.
- Stored versus Derived Attributes:

  In some cases, two (or more) attribute values are related such as the Age and Birth_date attributes of a person. For a particular person entity, the value of Age can be determined from the current (today's) date and the value of that person's Birth_date. The Age attribute is hence called a **derived attribute** and is said to be **derivable from** the Birth_date attribute, which is called a **stored attribute.**

- Complex Attributes:
  - Composite and Multivalued attributes can be nested arbitrarily.Such attributes are called as complex attributes.
  - Composite attributes can be represented by grouping components of a composite attribute between parentheses () and separating the components with commas, and by displaying multivalued attributes between braces { }.

- For example, if a person can have more than one residence and each residence can have a single address and multiplephones, an attribute Address_phone for a person can be specified as shown in below Figure 7.5. Both Phone and Address are themselves composite attributes.

{Address_phone({Phone(Area_code,Phone_number)},Address(Street_address (Number,Street,Apartment_number),City,State,Zip))}

Figure 7.5
A complex attribute: Address_phone.

- NULL Values
  - In some cases, a particular entity may not have an applicable value for an attribute. For example, a College_degrees attribute of a person entity applies only to people with college degrees. For such situations, a special value called NULL is created. a person with no college degree would have NULL for College_degrees.here meaning of NULL is not applicable.
  - NULL can also be used if we do not know the value of an attribute. For a particular entity such as employee, if we do not know the home phone number of an employee we can use NULL value. The meaning of NULL here is **unknown**.
  - The **unknown** category of NULL can be further classified into two cases. The first case arises when it is known that the attribute value exists but is **missing**. For instance, if the Height attribute of a person is listed as NULL. The second case arises when it is not known whether the attribute value exists, for example, if the Home_phone attribute of a person is NULL.

ENTITY TYPES, ENTITY SETS, KEYS, AND VALUE SETS

Entity Types and Entity Sets

- An **entity type** defines a *collection* (or *set*) of entities that have the same attributes. Each entity type in the database is described by its name and attributes.
- For example, a company employing hundreds of employees may want to store similar information concerning each of the employees. These employee entities share the same attributes, but each entity has its *own value*(*s*) for each attribute.
- The collection of all entities of a particular entity type in the database at any point in time is called an **entity set**. The entity set is usually referred to using the same name as the entity type. For example, EMPLOYEE refers to both a type of entity as well as the current set *of all employee entities* in the database.
- An entity type describes the **schema** or **intension** for a set of entities that share the same structure. The collection of entities of a particular entity type is grouped into an entity set, which is also called the **extension** of the entity type.
- The below Figure 7.6 shows two entity types: EMPLOYEE and COMPANY, and a list of some of the attributes for each Figure 7.6 shows two entity types: EMPLOYEE and COMPANY, and a list of some of the attributes for each. A few individual entities of each type are also illustrated, along with the values of their attributes.

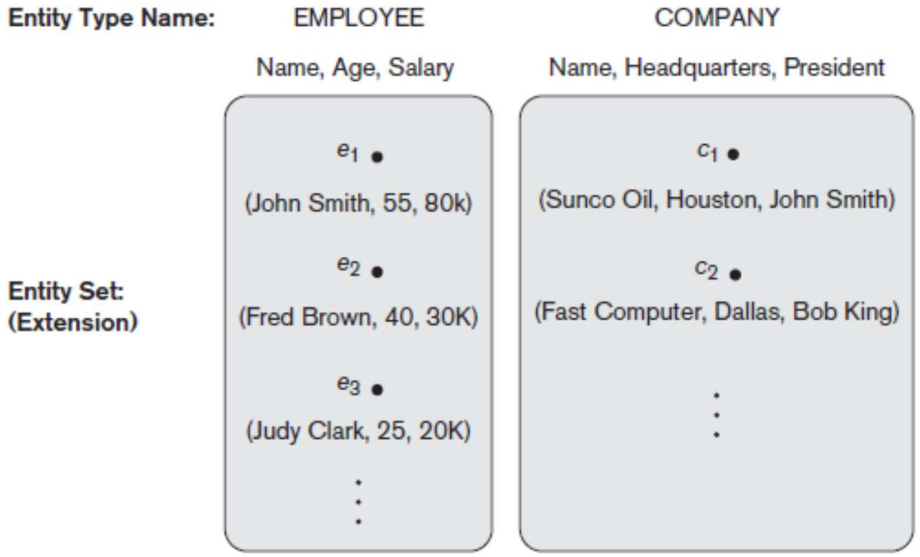


Figure 7.6
Two entity types, EMPLOYEE and COMPANY, and some member entities of each.

Key Attributes of an Entity Type

- An important constraint on the entities of an entity type is the **key** or **uniqueness constraint** on attributes. An entity type usually has one or more attributes whose values are distinct for each individual entity in the entity set. Such an attribute is called a **key attribute**, and its values can be used to identify each entity uniquely.

- For example, the Name attribute is a key of the COMPANY entity type in above Figure 7.6 because no two companies are allowed to have the same name. For the PERSON entity type, a typical key attribute is SSN (Social Security number).

- Sometimes several attributes together form a key, meaning that the combination of the attribute values must be distinct for each entity. If a set of attributes possesses this property, the proper way to represent this in the ER model is to define a composite attribute and designate it as a key attribute of the entity type.

- In ER diagrammatic notation, each key attribute has its name **underlined** inside the oval, as illustrated in below Figure 7.7(a). In ER notation, if two attributes are underlined separately, then each is a key on its own.

- Some entity types have more than one key attribute. For example, each of the Vehicle_id and Registration attributes of the entity type CAR in below Figure 7.7 is a key in its own right.

- The Registration attribute in below Figure 7.7 is an example of a composite key formed from two simple component attributes, State and Number, neither of which is a key on its own.

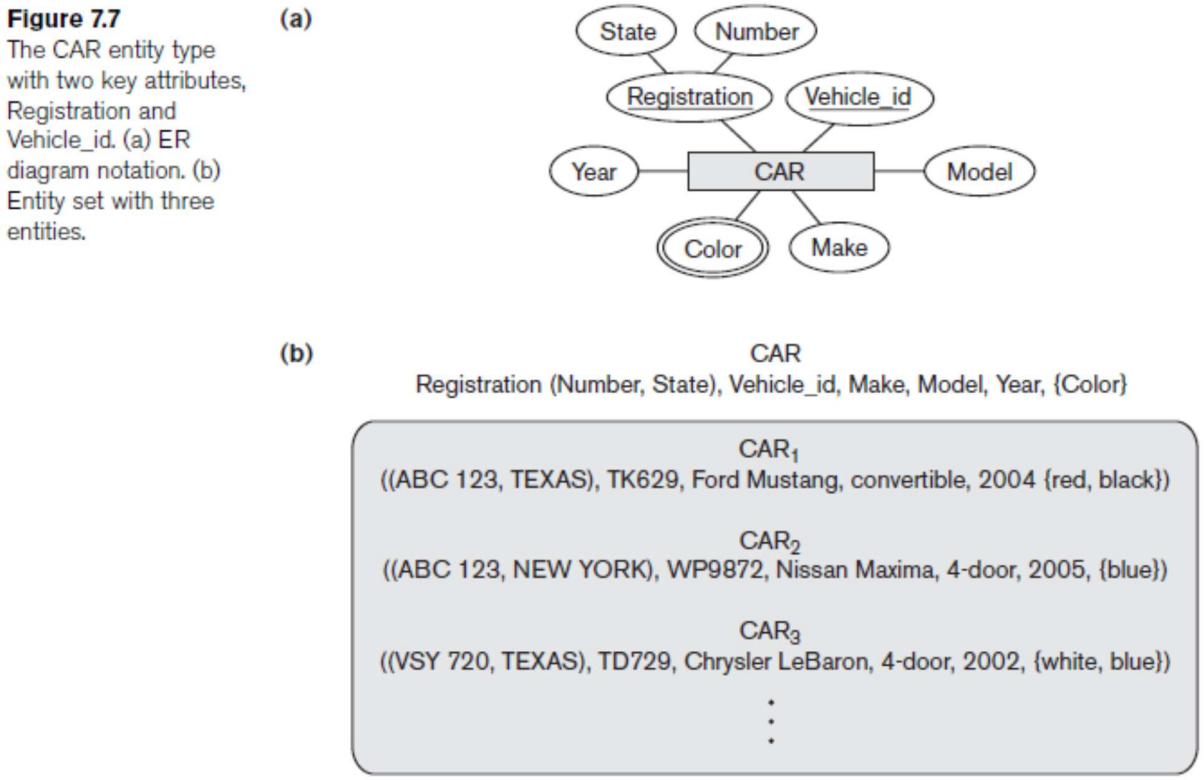


Figure 7.7
The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

Value Sets (Domains) of Attributes

- Each simple attribute of an entity type is associated with a **value set** (or **domain** of values), which specifies the set of values that may be assigned to that attribute for each individual entity.
- For example if the range of ages allowed for employees is between 16 and 70, we can specify the value set of the Age attribute of EMPLOYEE to be the set of integer numbers between 16 and 70.
- Value sets are not displayed in ER diagrams, and are typically specified using the basic **data types** available in most programming languages, such as integer, string, Boolean, float, enumerated type, subrange, and so on.

- Mathematically, an attribute *A* of entity set *E* whose value set is *V* can be defined as a **function** from *E* to the power set6 *P*(*V*) of *V*:

  $A : E \rightarrow P(V)$

- The value of attribute A for an entity e is represented as as A(e). A NULL value is represented by the *empty set*. For single-valued attributes, $A(e)$ is restricted to being a *singleton set* for each entity e in E, whereas there is no restriction on multivalued attributes.
- For a composite attribute A, the value set V is the power set of the Cartesian product of $P(V1)$, $P(V2)$, ..., $P(Vn)$, where $V1$, $V2$, ..., Vn are the value sets of the simple component attributes that form A:
  $V = P(P(V1) \times P(V2) \times ... \times P(Vn))$

INITIAL CONCEPTUAL DESIGN OF THE COMPANY DATABASE

- Figure 7.8 shows initial conceptual schema for the company database with 4 entity types DEPARTMENT, PROJECT, EMPLOYEE and DEPENDENT.
- An entity type DEPARTMENT with attributes Name, Number, Locations, Manager, and Manager_start_date. Locations is the only multivalued attribute. We can specify that both Name and Number are (separate) key attributes because each was specified to be unique.
- An entity type PROJECT with attributes Name, Number, Location, and Controlling_department. Both Name and Number are (separate) key attributes.
- An entity type EMPLOYEE with attributes Name, Ssn, Sex, Address, Salary, Birth_date, Department, and Supervisor. Both Name and Address may be composite attributes
- An entity type DEPENDENT with attributes Employee, Dependent_name, Sex, Birth_date, and Relationship (to the employee).

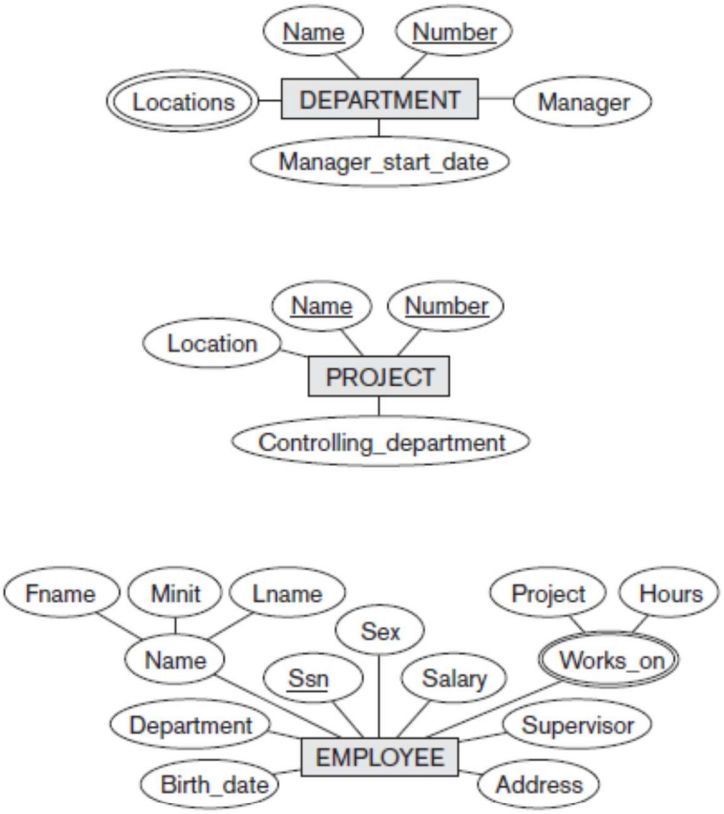


Figure 7.8
Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Review Questions

1. What is an attribute? Explain the different types of attributes that occur in ER-Model with Examples. Write their corresponding notations? (14 marks)
2. Discuss the conventions for displaying the ER schema as an ER diagram. (04 marks)
3. List out the notations for ER-diagram and their meaning. (10 marks)
4. With a neat diagram explain the phases of database design. (08 marks)

Session 7: RELATIONSHIP TYPES, RELATIONSHIP SETS, ROLES, AND STRUCTURAL CONSTRAINTS

- In the real world there are several implicit relationships among the various entity types. whenever an attribute of one entity type refers to another entity type, some relationship exists. For example, in a company database the attribute Manager of DEPARTMENT refers to an employee who manages the department and the attribute Controlling_department of PROJECT refers to the department that controls the project and so on.
- In the ER model, these references should not be represented as attributes but as **relationships**.

Relationship Types, Sets, and Instances

- A **relationship type** *R* among *n* entity types *E*1, *E*2, ..., *En* defines a set of associations or a **relationship set**, among entities from these entity types. A relationship type and its corresponding relationship set are customarily referred to by the same name, *R*. Mathematically, the relationship set *R* is a set of **relationship instances** ri*,* where each ri associates n individual entities (*e*1, *e*2, ..., *en*), and each entity ej in ri is a member of entity set *Ej*, 1 <= *j*<= *n*. Hence, a relationship set is a mathematical relation on *E*1, *E*2, ..., *En*. Each of the entity types *E*1, *E* 2, ..., *En* is said to participate in the relationship type *R*; similarly, each of the individual entities *e*1, *e*2, ..., *en* is said to **participate** in the relationship instance *ri* = (*e*1, *e*2, ..., *en*).
- For example, consider a relationship type WORKS_FOR between the two entity types EMPLOYEE and DEPARTMENT, which associates each employee with the department for which the employee works in the corresponding entity set. Each relationship instance in the relationship set WORKS_FOR associates one EMPLOYEE entity and one DEPARTMENT entity.
- Figure 7.9 below illustrates this example, where each relationship participate in *ri*. In the miniworld represented by Figure 7.9, employees *e*1, *e*3, and *e*6 work for department *d*1; employees *e*2 and *e*4 work for department *d*2; and employees *e*5 and *e*7 work for department *d*3.

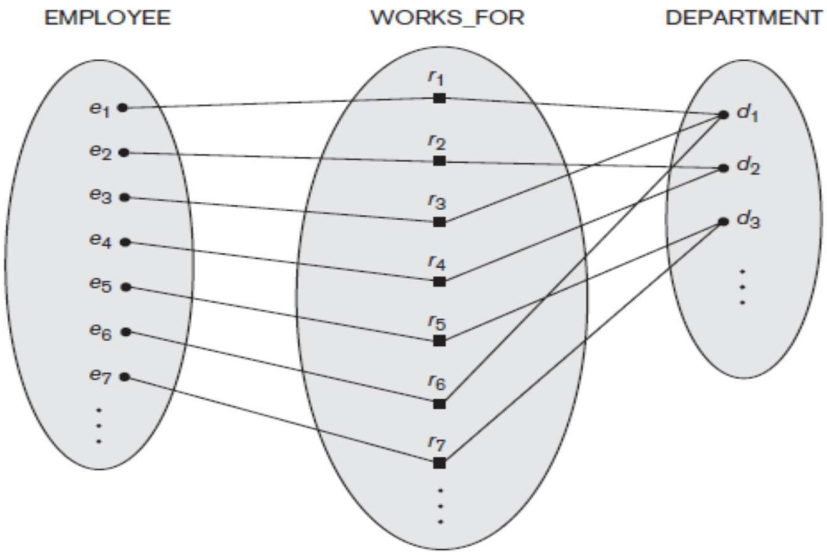


Figure 7.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Relationship Degree, Role Names, and Recursive Relationships

- **The Degree of a Relationship Type** is the number of participating entity types. For example, the WORKS_FOR relationship type is of degree two since two entity types EMPLOYEE and DEPARTMENT participate.
- A relationship type of degree two is called **binary**, and one of degree three is called **ternary**.
- **An example** of a ternary relationship is SUPPLY, shown in Figure in below 7.10, where each relationship instance *ri* associates three entities—a supplier *s*, a part *p*, and a project *j,* whenever *s* supplies part *p* to project *j*.
- Relationships can generally be of any degree, but the ones most common are binary relationships. Higherdegree relationships are generally more complex than binary relationships.

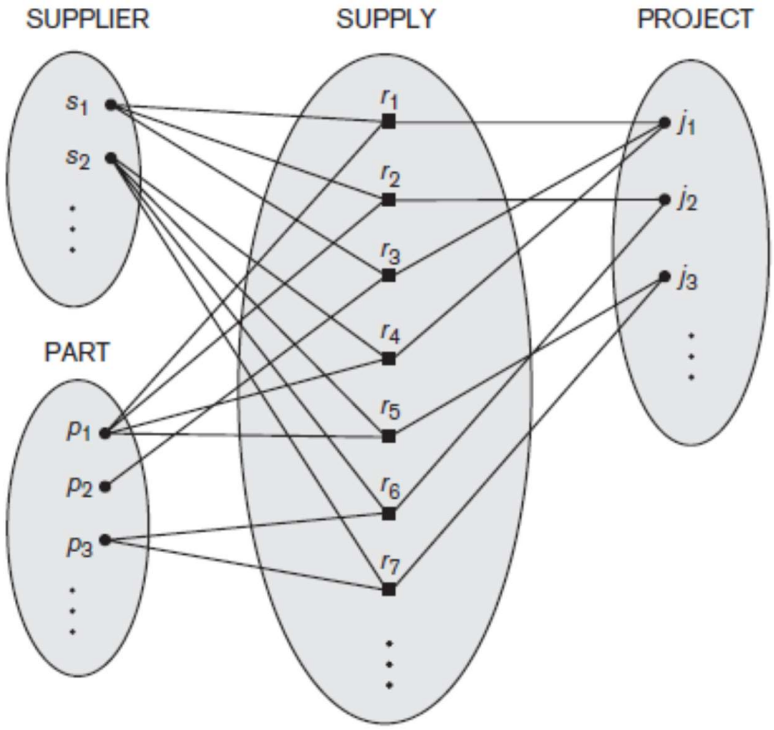


Figure 7.10
Some relationship instances in the SUPPLY ternary relationship set.

Relationships as Attributes

It is sometimes convenient to think of a binary relationship type in terms of attributes. Consider the WORKS_FOR relationship type in Figure 7.9. One can think of an attribute called Department of the EMPLOYEE entity type, where the value of Department for each EMPLOYEE entity is (a reference to) the DEPARTMENT entity for which that employee works. Hence, the value set for this Department attribute is the set of *all* DEPARTMENT entities, which is the DEPARTMENT entity set. However, when we think of a binary relationship as an attribute, we always have two options. In this example, the alternative is to think of a multivalued attribute Employee of the entity type DEPARTMENT whose values for each DEPARTMENT entity is the set of EMPLOYEE entities who work for that department. The value set of this Employee attribute is the power set of the EMPLOYEE entity set. Either of these two attributes, Department of EMPLOYEE or Employee of DEPARTMENT can represent the WORKS_FOR relationship type.

Role Names and Recursive Relationships

- Each entity type that participates in a relationship type plays a particular role in the relationship. The **role name** signifies the role that a participating entity from the entity type plays in each relationship instance
- For example, in the WORKS_FOR relationship type, EMPLOYEE plays the role of *employee* or *worker* and DEPARTMENT plays the role of *department* or *employer*.
- In some cases the *same* entity type participates more than once in a relationship type in *different roles*. In such cases the role name becomes essential for distinguishing the meaning of the role that each participating entity plays. Such relationship types are called **recursive relationships**.
- The below Figure 7.11 shows an example of recursive relationship. The SUPERVISION relationship type relates an employee to a supervisor, where both employee and supervisor entities are members of the same EMPLOYEE entity set. Hence, the EMPLOYEE entity type *participates twice* in SUPERVISION: once in the role of *supervisor* (or *boss*), and once in the role of *supervisee* (or *subordinate*). In Figure 7.11, the lines marked '1' represent the supervisor role, and those marked '2' represent the supervisee role.

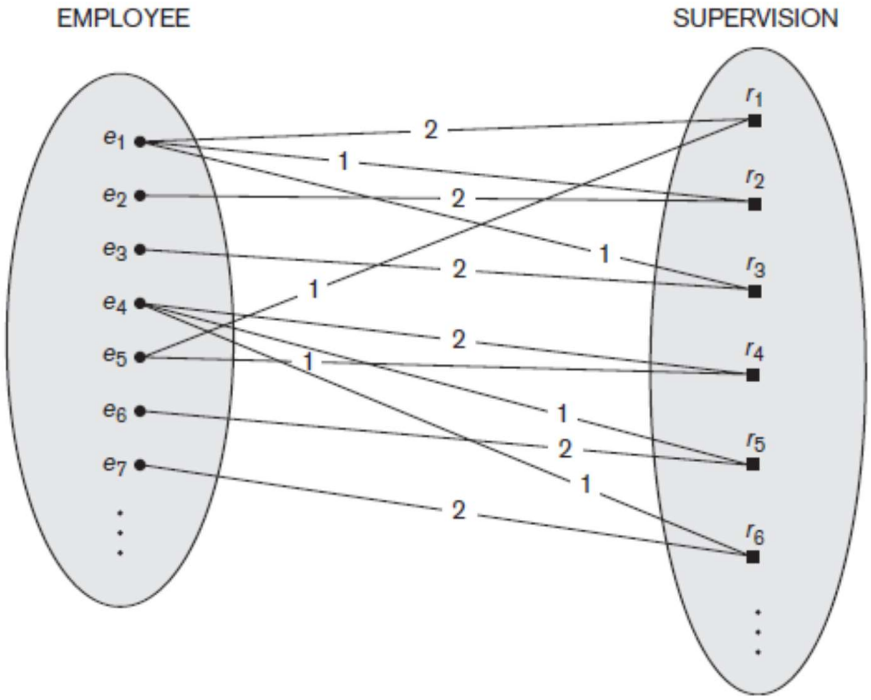


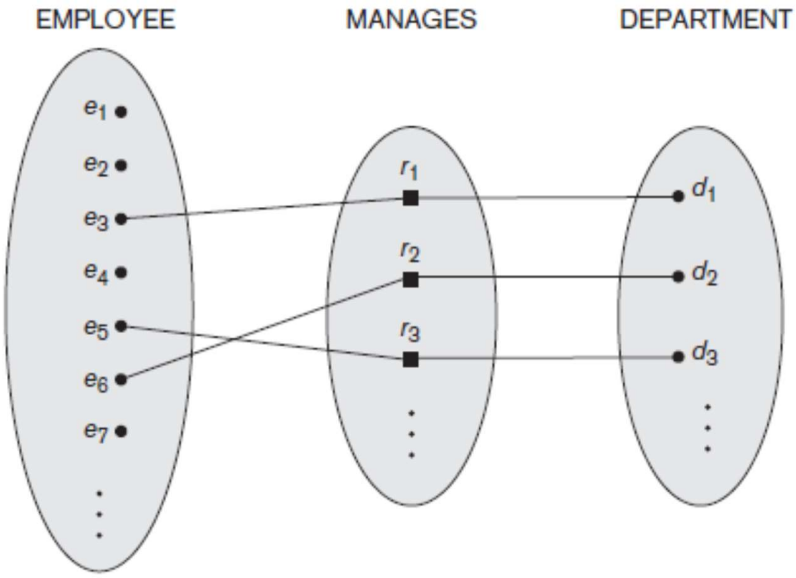
Figure 7.11
A recursive relationship SUPERVISION between EMPLOYEE in the *supervisor* role (1) and EMPLOYEE in the *subordinate* role (2).

Constraints on Binary Relationship Types

- Relationship types usually have certain constraints that limit the possible combinations of entities that may participate in the corresponding relationship set.
- For example, if the company has a rule that each employee must work for exactly one department, then we would like to describe this constraint in the schema.
- There are two main types of binary relationship constraints: *cardinality ratio* and *participation*.
- The **cardinality ratio** for a binary relationship specifies the *maximum* number of relationship instances that an entity can participate in.
- For example, in the WORKS_FOR binary relationship type, DEPARTMENT:EMPLOYEE is of cardinality ratio 1:N, meaning that each department can be related to (that is, employs) any number of employees, but an employee can be related to (work for) only one department.
- The possible cardinality ratios for binary relationship types are 1:1, 1:N, N:1, and M:N.

- An example of a 1:1 binary relationship is MANAGES shown in below Figure 7.12, which relates a department entity to the employee who manages that department. This represents the miniworld constraints that at any point in time an employee can manage one department only and a department can have one manager only.

Figure 7.12
A 1:1 relationship, MANAGES.



- The relationship type WORKS_ON shown in below Figure 7.13 is of cardinality ratio M:N, because the mini- world rule is that an employee can work on several projects and a project can have several employees.

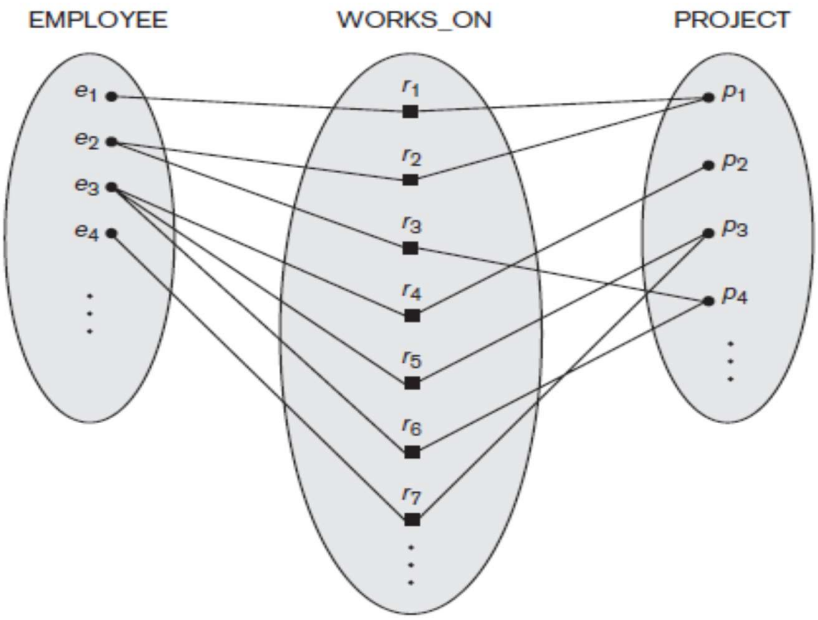


Figure 7.13
An M:N relationship, WORKS_ON.

Participation Constraints and Existence Dependencies

- The **participation constraint** specifies whether the existence of an entity depends on its being related to another entity via the relationship type. This constraint specifies the *minimum* number of relationship instances that each entity can participate in, and is sometimes called the **minimum cardinality constraint**.
- There are two types of participation constraints total and partial.
- For example If a company policy states that *every* employee must work for a department, then an employee entity can exist only if it participates in at least one WORKS_FOR relationship instance (Figure 7.9). Thus, the participation of EMPLOYEE in WORKS_FOR is called **total participation**, meaning that every entity in *the total set* of employee entities must be related to a department entity via WORKS_FOR. Total participation is also called **existence dependency**.
- In Figure 7.12 we do not expect every employee to manage a department, so the participation of EMPLOYEE in the MANAGES relationship type is **partial**, meaning that

some or *part of the set of* employee entities are related to some department entity via MANAGES, but not necessarily all.

- The cardinality ratio and participation constraints, taken togetheris refered as the **structural constraints** of a relationship type.
- In ER diagrams, total participation (or existence dependency) is displayed as a *double line* connecting the participating entity type to the relationship, whereas partial participation is represented by a *single line* (see Figure 7.2).

Attributes of Relationship Types

- Relationship types can also have attributes, similar to those of entity types.
- For example, to record the number of hours per week that an employee works on a particular project, we can include an attribute Hours for the WORKS_ON relationship type in Figure 7.13. Another example is to include the date on which a manager started managing a department via an attribute Start_date for the MANAGES relationship type in Figure 7.12.
- The attributes of 1:1 relationship type can be migrated to one of the participating entity types.
- For example, the Start_date attribute for the MANAGES relationship can be an attribute of either EMPLOYEE or DEPARTMENT, although conceptually it belongs to MANAGES. This is because MANAGES is a 1:1 relationship, so every department or employee entity participates in *at most one* relationship instance. Hence, the value of the Start_date attribute can be determined separately, either by the participating department entity or by the participating employee (manager) entity.
- For a 1:N relationship type, a relationship attribute can be migrated *only* to the entity type on the N-side of the relationship. For example, in Figure 7.9, if the WORKS_FOR relationship also has an attribute Start_date that indicates when an employee started working for a department, this attribute can be included as an attribute of EMPLOYEE. This is because each employee works for only one department, and hence participates in at most one relationship instance in WORKS_FOR.

- For M:N relationship types, some attributes may be determined by the *combination of participating entities* in a relationship instance, not by any single entity. Such attributes *must be specified as relationship attributes*. An example is the Hours attribute of the M:N relationship WORKS_ON (Figure 7.13

Session 8: Weak Entity Types ER Diagrams Examples

Weak Entity Type

- Entity types that do not have key attributes of their own are called **weak entity types**. In contrast, **regular entity types** that do have a key attribute are called **strong entity types**. Entities belonging to a weak entity type are identified by being related to specific entities from another entity type in combination with one of their attribute values. This other entity type is the **identifying** or **owner entity type**, and the relationship type that relates a weak entity type to its owner is the **identifying relationship** of the weak entity type.
- A weak entity type always has a *total participation constraint* (existence dependency) with respect to its identifying relationship because a weak entity cannot be identified without an owner entity.
- Consider the entity type DEPENDENT, related to EMPLOYEE, which is used to keep track of the dependents of each employee via a 1:N relationship (Figure 7.2). DEPENDENT is the weak entity type, EMPLOYEE is the owner Entity type and DEPENDENTS of is the identifying relationship.
- A weak entity type normally has a **partial key**, which is the attribute that can uniquely identify weak entities that are *related to the same owner entity*. for example, if we assume that no two dependents of the same employee ever have the same first name, the attribute Name of DEPENDENT is the partial key.
- In general, any number of levels of weak entity types can be defined. an owner entity type may itself be a weak entity type. In addition, a weak entity type may have more than one identifying entity type and an identifying relationship type of degree higher than two.

Refining the ER Design for the COMPANY Database

- The database design in Figure 7.8 can be refined by changing the attributes that represent relationships into relationship types.
- we specify the following relationship types:
- MANAGES, a 1:1 relationship type between EMPLOYEE and DEPARTMENT. EMPLOYEE participation is partial. DEPARTMENT participation is not clear from the requirements. We question the users, who say that a department must have a manager at all times, which implies total participation. The attribute Start_date is assigned to this relationship type.
- WORKS_FOR, a 1:N relationship type between DEPARTMENT and EMPLOYEE. Both participations are total.
- CONTROLS, a 1:N relationship type between DEPARTMENT and PROJECT. The participation of PROJECT is total, whereas that of DEPARTMENT is determined to be partial, after consultation with the users indicates that some departments may control no projects.
- SUPERVISION, a 1:N relationship type between EMPLOYEE (in the supervisor role) and EMPLOYEE (in the supervisee role). Both participations are determined to be partial, after the users indicate that not every employee is a supervisor and not every employee has a supervisor.
- WORKS_ON, determined to be an M:N relationship type with attribute Hours, after the users indicate that a project can have several employees working on it. Both participations are determined to be total.
- DEPENDENTS_OF, a 1:N relationship type between EMPLOYEE and DEPENDENT, which is also the identifying relationship for the weak entity DEPENDENT is total.

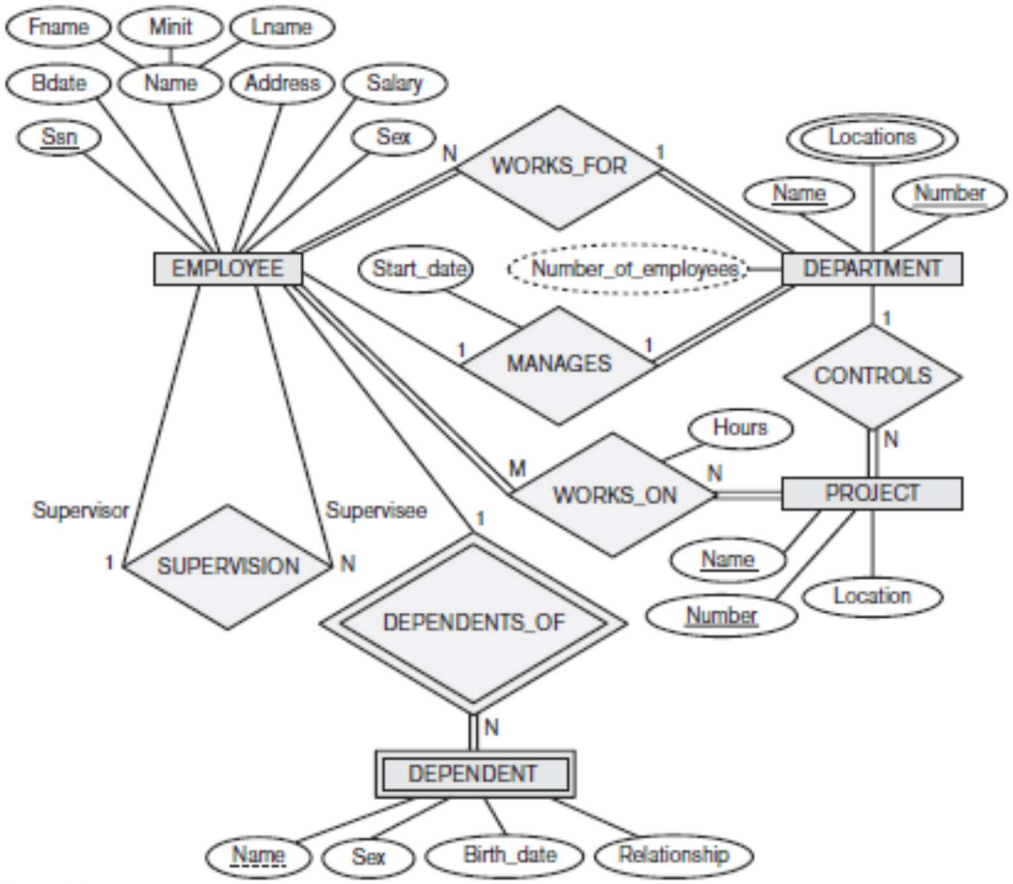


Figure 7.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 7.14.

ER Diagrams, Naming Conventions, and Design Issues

Summary of Notation for ER Diagrams

- Figures 7.9 through 7.13 illustrate examples of the participation of entity types in relationship types by displaying their sets or extensions—the individual entity instances in an entity set and the individual relationship instances in a relationship set.
- Figure 7.2 displays the COMPANY **ER database schema** as an **ER diagram**. Entity types such as EMPLOYEE, DEPARTMENT, and PROJECT are shown in rectangular boxes.
- Relationship types such as WORKS_FOR, MANAGES, CONTROLS, and WORKS_ON are shown in diamond-shaped boxes attached to the participating entity types with straight lines.

- Attributes are shown in ovals, and each attribute is attached by a straight line to its entity type or relationship type.
- Component attributes of a composite attribute are attached to the oval representing the composite attribute, as illustrated by the Name attribute of EMPLOYEE. Multivalued attributes are shown in double ovals, as illustrated by the Locations attribute of DEPARTMENT. Key attributes have their names underlined. Derived attributes are shown in dotted ovals, as illustrated by the Number_of_employees attribute of DEPARTMENT. Weak entity types are distinguished by being placed in double rectangles and by having their identifying relationship placed in double diamonds, as illustrated by the DEPENDENT entity type and the DEPENDENTS_OF identifying relationship type. The partial key of the weak entity type is underlined with a dotted line.
- In Figure 7.2 the cardinality ratio of each *binary* relationship type is specified by attaching a 1, M, or N on each participating edge. The cardinality ratio of DEPARTMENT:EMPLOYEE in MANAGES is 1:1, whereas it is 1:N for DEPARTMENT: EMPLOYEE in WORKS_FOR, and M:N for WORKS_ON. The participation for total participation (existence dependency). In Figure 7.2 we show the role names for the SUPERVISION relationship type because the same EMPLOYEE entity type plays two distinct roles in that relationship. Notice that the cardinality ratio is 1:N from supervisor to supervisee because each employee in the role of supervisee has at most one direct supervisor, whereas an employee in the role of supervisor can supervise zero or more employees.

Proper Naming of Schema Constructs

- When designing a database schema, the choice of names for entity types, attributes, relationship types, and (particularly) roles is not always straightforward. One should choose names that convey, as much as possible, the meanings attached to the different constructs in the schema.
- We choose to use *singular names* for entity types, rather than plural ones, because the entity type name applies to each individual entity belonging to that entity type. In our ER diagrams, we will use the convention that entity type and relationship type names are

uppercase letters, attribute names have their initial letter capitalized, and role names are lowercase letters.

- As a general practice, given a narrative description of the database requirements, the *nouns* appearing in the narrative tend to give rise to entity type names, and the *verbs* tend to indicate names of relationship types. Attribute names generally arise from additional nouns that describe the nouns corresponding to entity types.
- Another naming consideration involves choosing binary relationship names to make the ER diagram of the schema readable from left to right and from top to bottom.

Design Choices for ER Conceptual Design

- In general, the schema design process should be considered an iterative refinement process, where an initial design is created and then iteratively refined until the most suitable design is reached.
- Some of the refinements that are often used include the following:
  - A concept may be first modeled as an attribute and then refined into a relationship because it is determined that the attribute is a reference to another entity type. It is often the case that a pair of such attributes that are inverses of one another are refined into a binary relationship.
  - Similarly, an attribute that exists in several entity types may be elevated or promoted to an independent entity type. For example, suppose that several entity types in a UNIVERSITY database, such as STUDENT, INSTRUCTOR, and COURSE, each has an attribute Department in the initial design. the designer may then choose to create an entity type DEPARTMENT with a single attribute Dept_name and relate it to the three entity types (STUDENT, INSTRUCTOR, and COURSE) via appropriate relationships.
  - An inverse refinement to the previous case may be applied, for example, if an entity type DEPARTMENT exists in the initial design with a single attribute Dept_name and is related to only one other entity type, STUDENT. In this case, DEPARTMENT may be reduced or demoted to an attribute of STUDENT.

Alternative Notations for ER Diagrams

- There are many alternative diagrammatic notations for displaying ER diagrams such as Unified Modeling Language (UML) notation for class diagrams, which has been proposed as a standard for conceptual object modeling.
- One alternative ER notation is used for specifying structural constraints on relationships, which replaces the cardinality ratio (1:1, 1:N, M:N) and single/double line notation for participation constraints. This notation involves associating a pair of integer numbers (min, max) with each *participation* of an entity type *E* in a relationship type *R*, where $0 \leq \text{min} \leq \text{max}$ and $\text{max} \geq 1$.
- The numbers mean that for each entity *e* in *E*, *e* must participate in at least min and at most max relationship instances in *R at any point in time*. In this method, min = 0 implies partial participation, whereas min > 0 implies total participation.
- Figure 7.15 below displays the COMPANY database schema using the (min, max) notation. Usually, one uses either the cardinality ratio/single-line/double-line notation *or* the (min, max) notation. The (min, max) notation is more precise, and we can use it to specify some structural constraints for relationship types of *higher degree*.

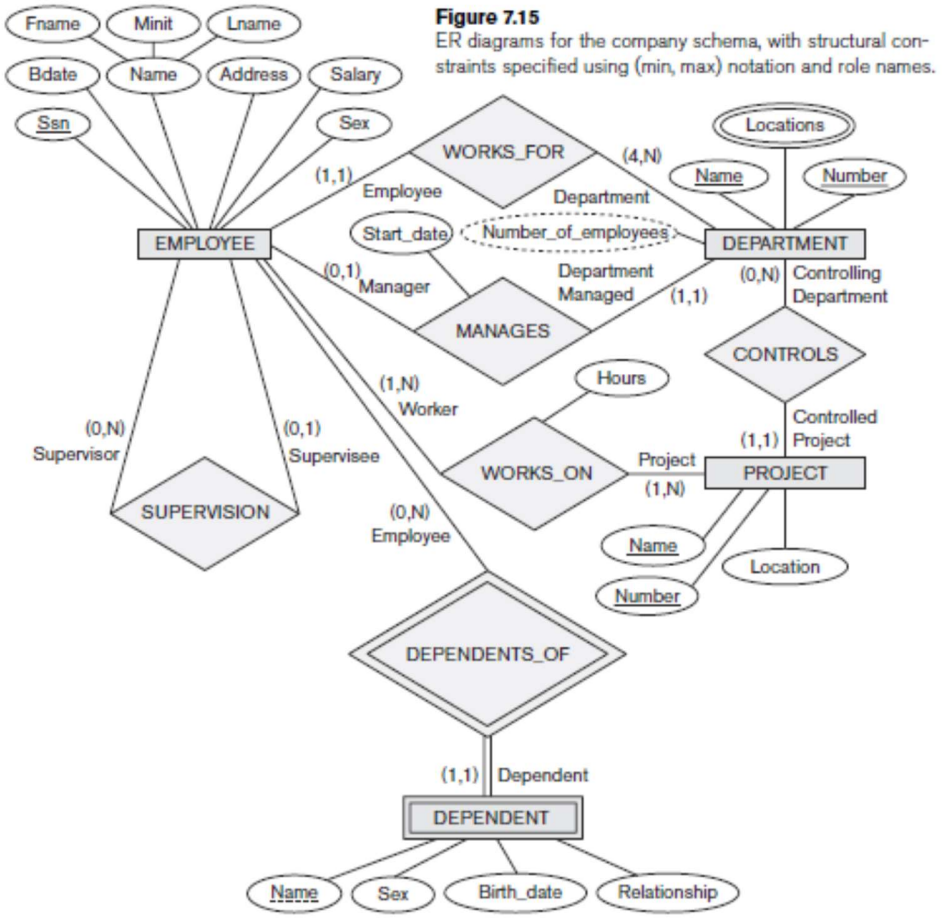


Figure 7.15
ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.

Example of Other Notation: UML Class Diagrams

- UML class diagrams can be considered as an alternative notation to ER diagrams.
- In UML class diagrams, a **class** (similar to an entity type in ER) is displayed as a box that includes three sections: The top section gives the **class name,** the middle section includes the **attributes** and the last section includes **operations** that can be applied to individual objects of the class.
- Consider the EMPLOYEE class in below Figure 3.16. Its attributes are Name, Ssn, Bdate, Sex, Address, and Salary. The designer can optionally specify the **domain** (or data type) of an attribute if desired, by placing a colon (:) followed by the domain name or description, as illustrated by the Name, Sex, and Bdate attributes of EMPLOYEE in Figure 3.16.

- A composite attribute is modelled as a **structured domain**, as illustrated by the Name attribute of EMPLOYEE. A multivalued attribute will generally be modelled as a separate class, as illustrated by the LOCATION class in Figure 3.16.

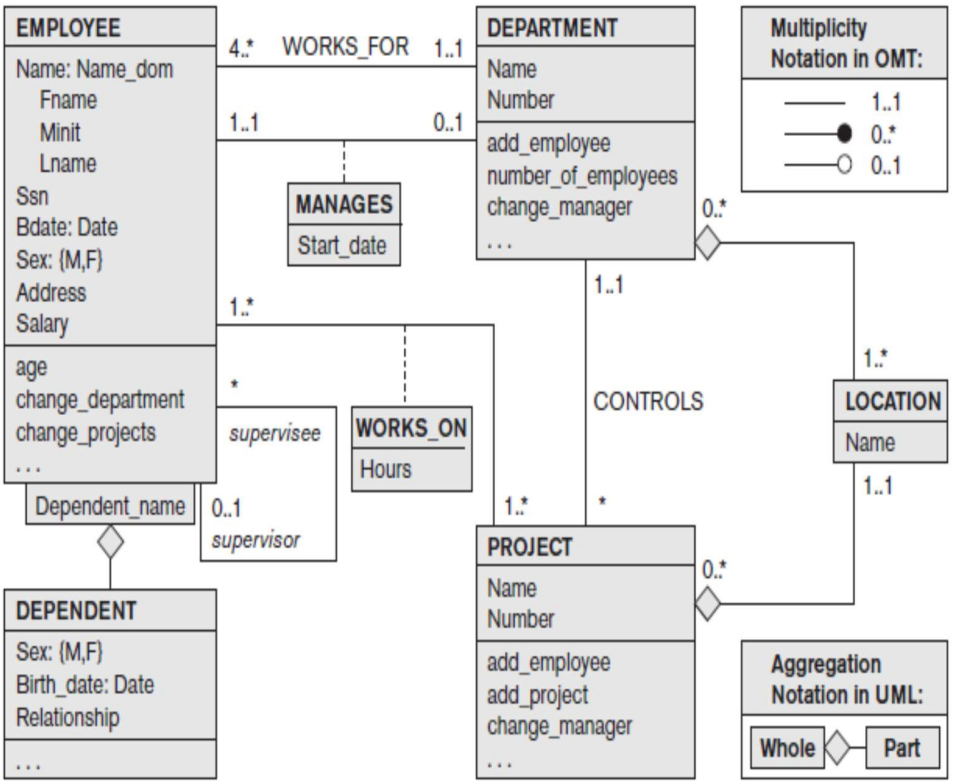


Figure 3.16
The COMPANY conceptual schema in UML class diagram notation.

Relationship Types of Degree Higher than Two

The **degree** of a relationship type is the number of participating entity types and a relationship type of degree two is ***binary*** and a relationship type of degree three is ***ternary*.**

Choosing between Binary and Ternary (or Higher-Degree) Relationships

- The ER diagram notation for a ternary relationship type is shown in below Figure 7.17(a), which displays the schema for the SUPPLY relationship type.

- The relationship set of SUPPLY is a set of relationship instances (*s*, *j*, *p*), where *s* is a SUPPLIER who is currently supplying a PART *p* to a PROJECT *j*. In general, a relationship type *R* of degree *n* will have *n* edges in an ER diagram, one connecting *R* to each participating entity type.

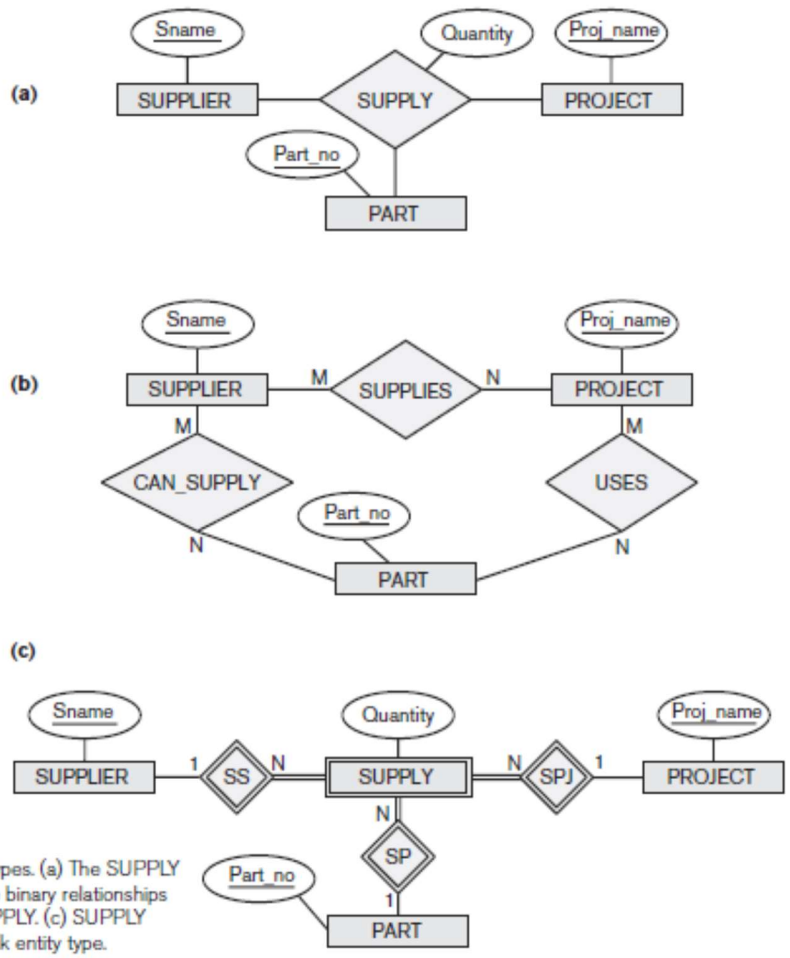


Figure 7.17
Ternary relationship types. (a) The SUPPLY relationship. (b) Three binary relationships not equivalent to SUPPLY. (c) SUPPLY represented as a weak entity type.

- Figure 7.17(b) shows an ER diagram for three binary relationship types CAN_SUPPLY, USES, and SUPPLIES. In general, a ternary relationship type represents different information than do three binary relationship types.
- Consider the three binary relationship types CAN_SUPPLY, USES, and SUPPLIES.
- Suppose that CAN_SUPPLY, between SUPPLIER and PART, includes an instance (*s*, *p*) whenever supplier *s can supply* part *p* (to any project).

- USES, between PROJECT and PART, includes an instance (*j*, *p*) whenever project *j* uses part *p*. SUPPLIES, between SUPPLIER and PROJECT, includes an instance (*s*, *j*) whenever supplier *s* supplies *some part* to project *j*.
- The existence of three relationship instances (*s*, *p*), (*j*, *p*), and (*s*, *j*) in CAN_SUPPLY, USES, and SUPPLIES, respectively, does not necessarily imply that an instance (*s*, *j*, *p*) exists in the ternary relationship SUPPLY, because the *meaning is different*. It is often tricky to decide whether a particular relationship should be represented as a relationship type of degree *n* or should be broken down into several relationship types of smaller degrees. The typical solution is to include the ternary relationship *plus* one or more of the binary relationships, if they represent different meanings and if all are needed by the application.
- Some database design tools are based on variations of the ER model that permit only binary relationships. In this case, a ternary relationship such as SUPPLY must be represented as a weak entity type, with no partial key and with three identifying relationships.
- The three participating entity types SUPPLIER, PART, and PROJECT are together the owner entity types as shown in below Figure 7.17(c).Hence, an entity in the weak entity type SUPPLY in Figure 7.17(c) is identified by the combination of its three owner entities from SUPPLIER, PART, and PROJECT.
- Another example is shown in Figure 7.18. The ternary relationship type OFFERS represents information on instructors offering courses during particular semesters, hence it includes a relationship instance (*i*, *s*, *c*) whenever INSTRUCTOR *i* offers COURSE *c* during SEMESTER *s*. The three binary relationship types shown in Figure 7.18 have the following meanings:
- CAN_TEACH relates a course to the instructors who *can teach* that course, TAUGHT_DURING relates a semester to the instructors who *taught some course* during that semester, and OFFERED_DURING relates a semester to the courses offered during that semester *by any instructor*.

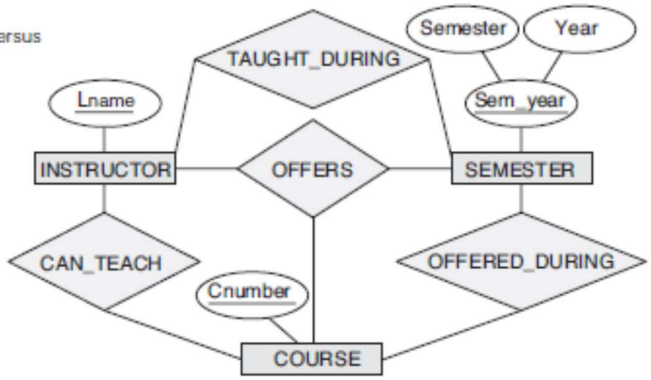


Figure 7.18
Another example of ternary versus binary relationship types.

- It is possible to have a weak entity type with a ternary (or *n*-ary) identifying relationship type. In this case, the weak entity type can have *several* owner entity types. An example is shown in Figure 7.19. This example shows part of a database that keeps track of candidates interviewing for jobs at various companies, and may be part of an employment agency database.
- For example. In the requirements, a candidate can have multiple interviews with the same company (for example, with different company departments or on separate dates), but a job offer is made based on one of the interviews. Here, INTERVIEW is represented as a weak entity with two owners CANDIDATE and COMPANY, and with the partial key Dept_date. An INTERVIEW entity is uniquely identified by a candidate, a company, and the combination of the date and department of the interview.

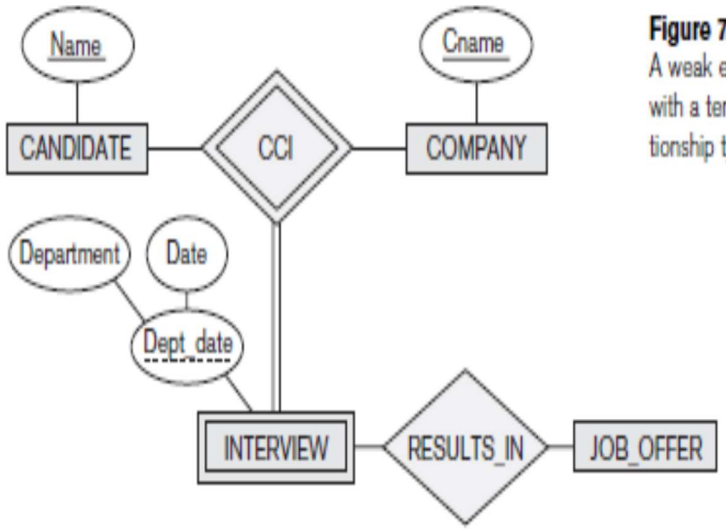


Figure 7.19
A weak entity type INTERVIEW with a ternary identifying relationship type.

Constraints on Ternary (or Higher-Degree) Relationships

- There are two notations for specifying structural constraints on *n*-ary relationships, and they specify different constraints.
- The first notation is based on the cardinality ratio notation of binary relationships. Here, a 1, M, or N is specified on each participation arc (both M and N symbols stand for *many* or *any number*).
- For example, In the SUPPLY relationship in Figure 7.17, Suppose that the constraint exists that for a particular project-part combination, only one supplier will be used. In this case, we place 1 on the SUPPLIER participation, and M, N on the PROJECT, PART participations in cardinality ratio notation. This specifies the constraint that a particular (*j*, *p*) combination can appear at most once in the relationship set because each such (PROJECT, PART) combination uniquely determines a single supplier.
- The second notation is based on the (min, max) notation displayed in Figure 7.15 for binary relationships. A (min, max) on a participation here specifies that each entity is related to at least *min* and at most *max relationship instances* in the relationship set. These constraints have no bearing on determining the key of an *n*-ary relationship, where *n* > 2,17 but specify a different type of constraint that places restrictions on how many relationship instances each entity can participate in.

SPECIALIZATION AND GENERALIZATION

Specialization

- **Specialization** is the process of defining a *set of subclasses* of an entity type, this entity type is called the **superclass** of the specialization. The set of subclasses that forms a specialization is defined on the basis of some distinguishing characteristic of the entities in the superclass.
- For example, the set of subclasses {SECRETARY, ENGINEER, TECHNICIAN} is a specialization of the superclass EMPLOYEE that distinguishes among employee entities based on the *job type* of each employee.

- Below Figure 4.2 shows a few entity instances that belong to subclasses of the {SECRETARY, ENGINEER, TECHNICIAN} specialization.

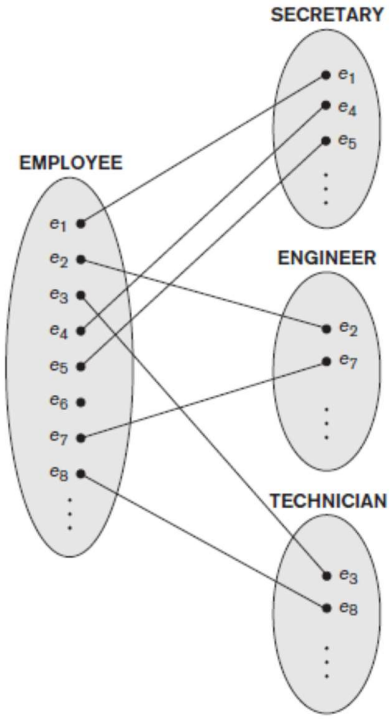


Figure 4.2
Instances of a specialization.

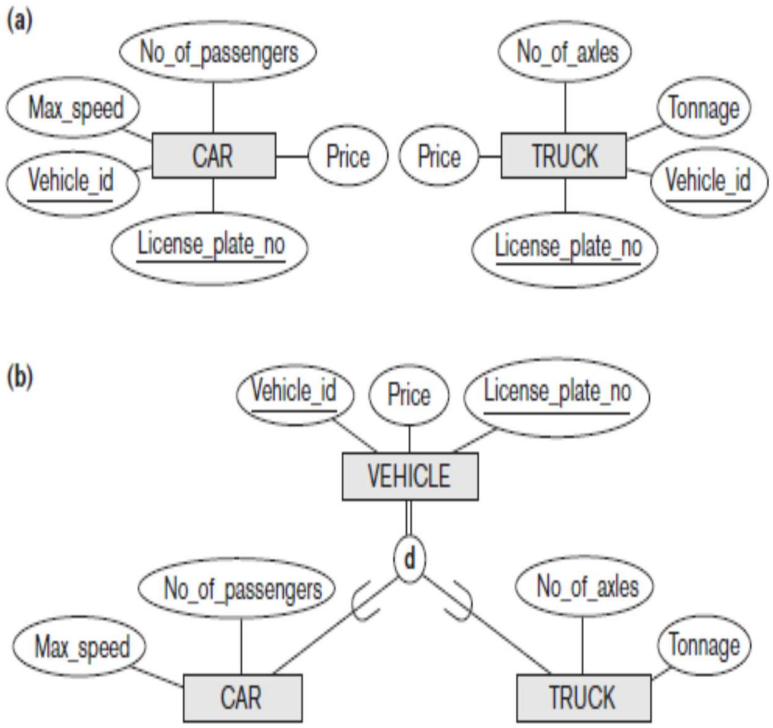
There are two main reasons for including class/subclass relationships and specializations.

- The first is that certain attributes may apply to some but not all entities of the superclass entity type. . A subclass is defined in order to group the entities to which these attributes apply. The members of the subclass may still share the majority of their attributes with the other members of the superclass.
- The second reason for using subclasses is that some relationship types may be participated in only by entities that are members of the subclass.

Generalization

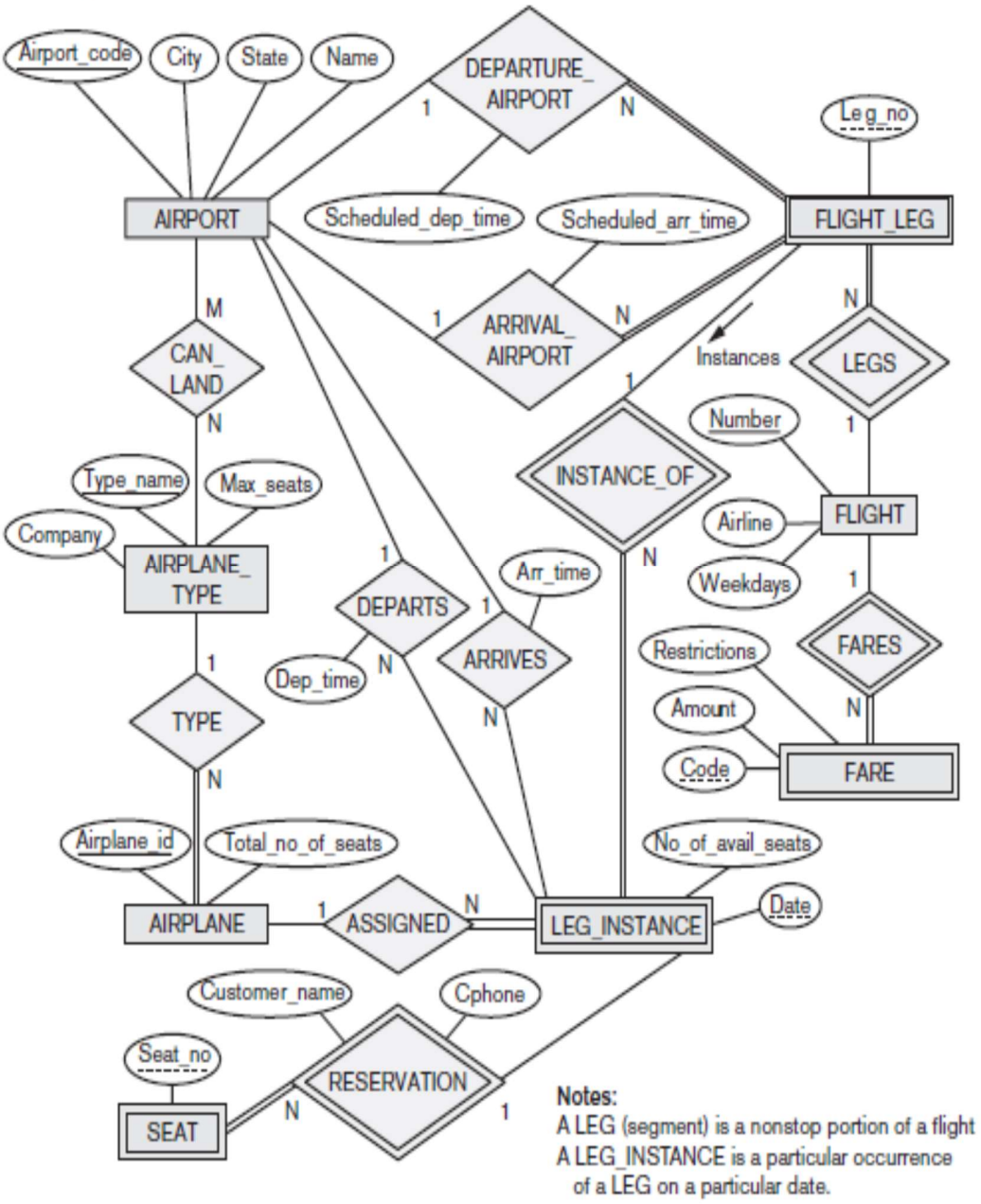
- Generalization is a *reverse process* of abstraction in which suppresses the differences among several entity types, identify their common features, and **generalize** them into a single **superclass** of which the original entity types are special **subclasses**.
- For example, consider the entity types CAR and TRUCK shown in Figure 4.3(a). Because they have several common attributes, they can be generalized into the entity type VEHICLE, as shown in Figure 4.3(b).
- Both CAR and TRUCK are now subclasses of the **generalized superclass** VEHICLE. The term **generalization** to refer to the process of defining a generalized entity type from the given entity types.

Figure 4.3
Generalization. (a) Two entity types, CAR and TRUCK.
(b) Generalizing CAR and TRUCK into the superclass VEHICLE.



ER-DIAGRAM FOR AIRLINE DATABASE

Figure 7.20
An ER diagram for an AIRLINE database schema.



ER-DIAGRAM FOR BANK DATABASE

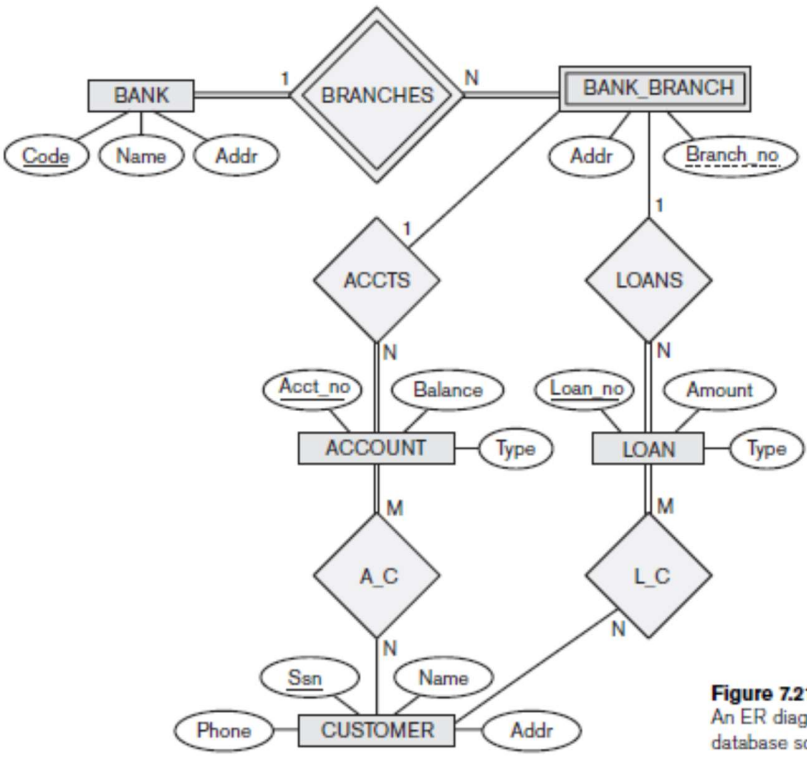
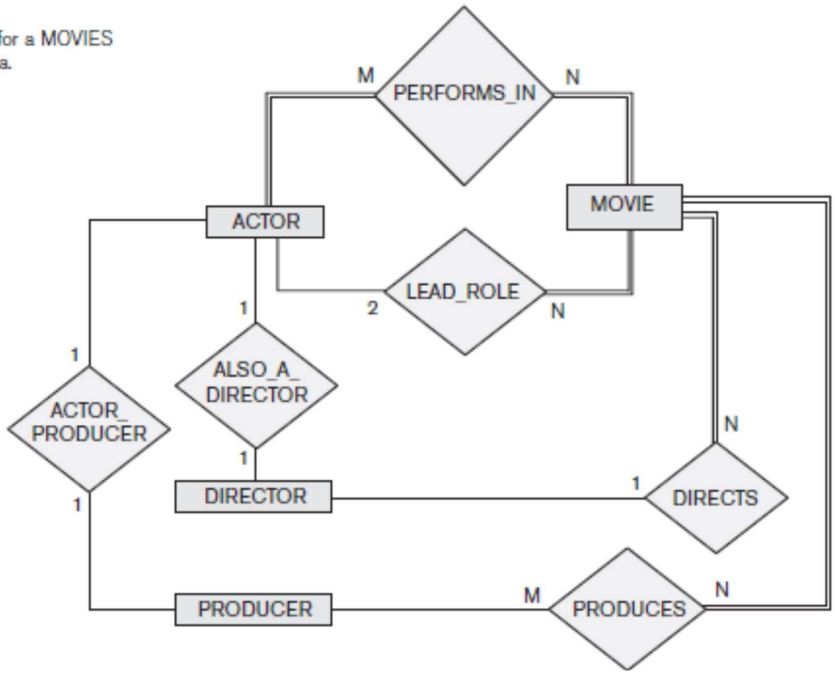


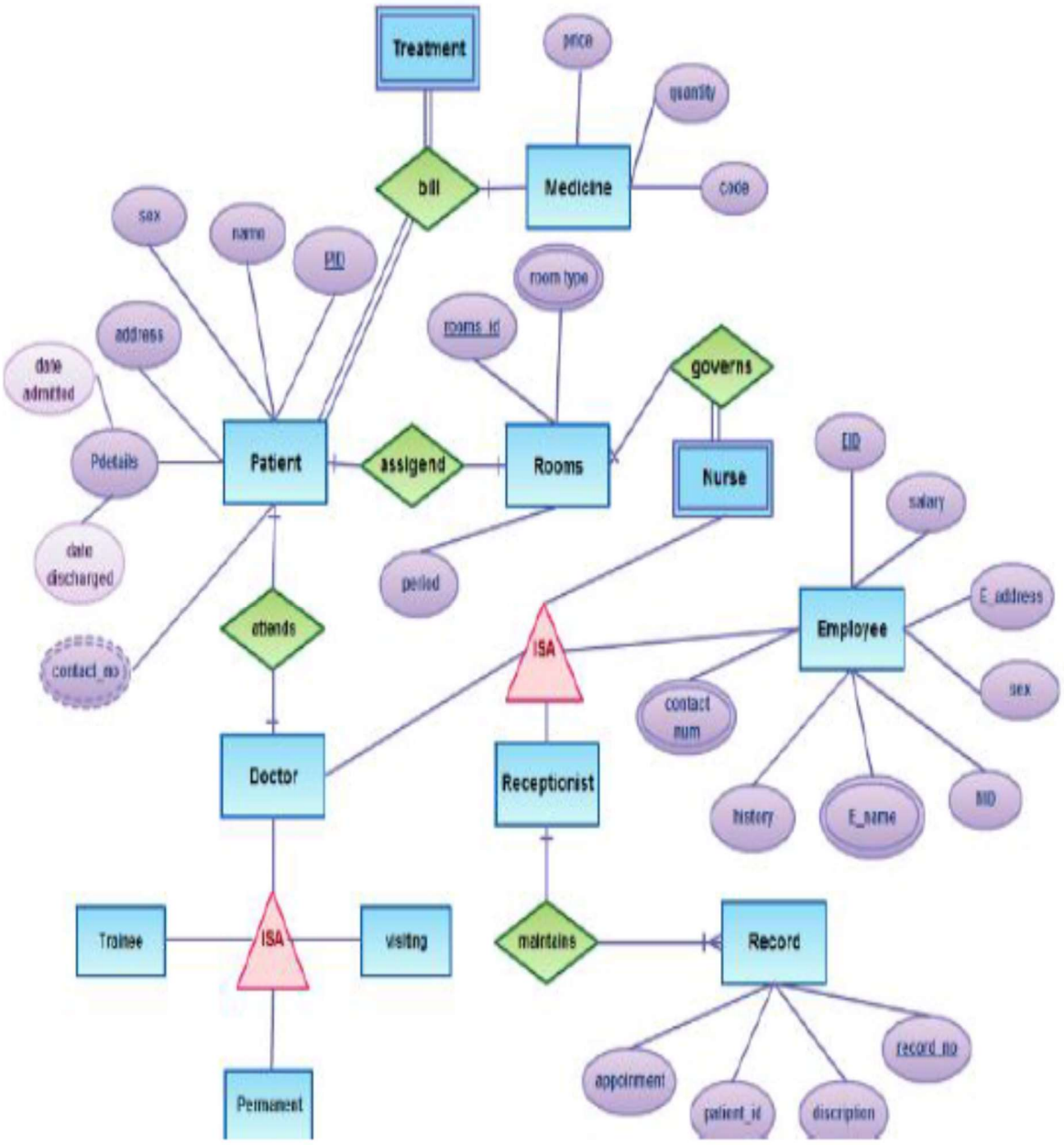
Figure 7.21
An ER diagram for a BANK database schema.

ER-DIAGRAM FOR A MOVIES DATABASE SCHEMA

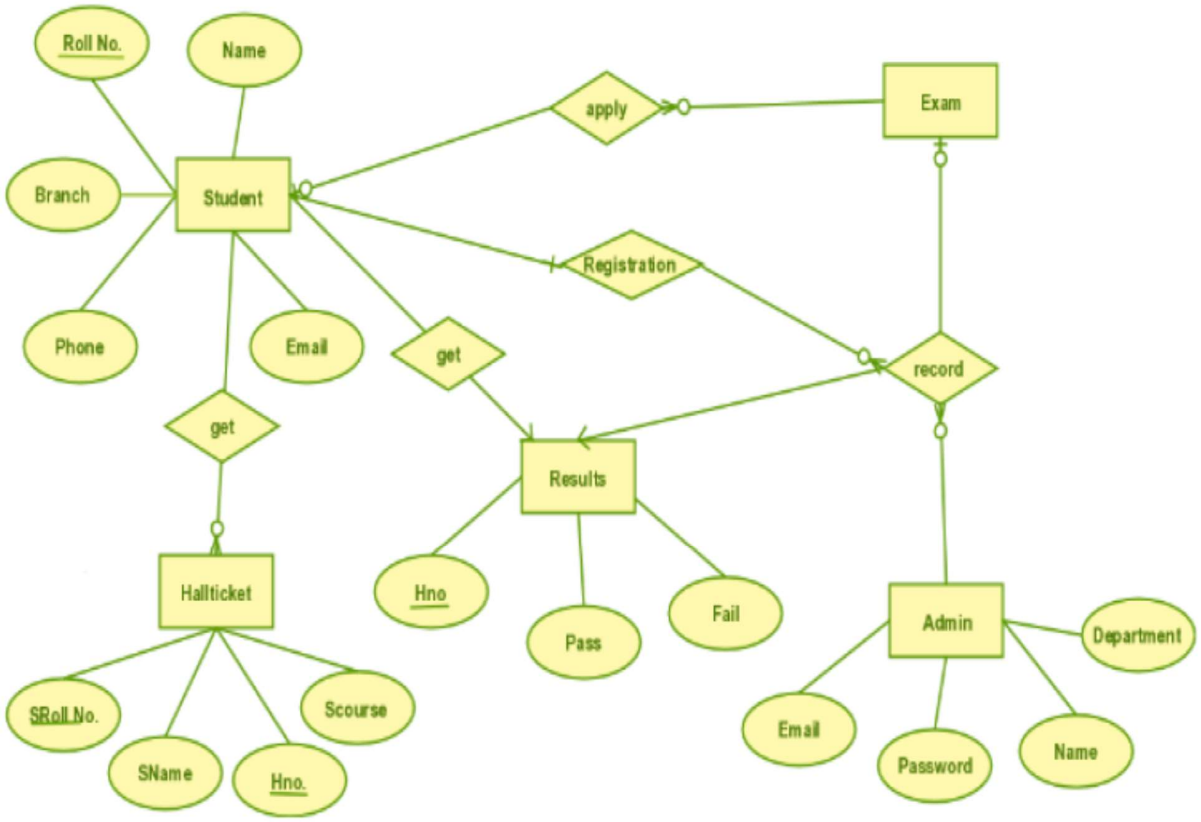
Figure 7.24
An ER diagram for a MOVIES database schema.



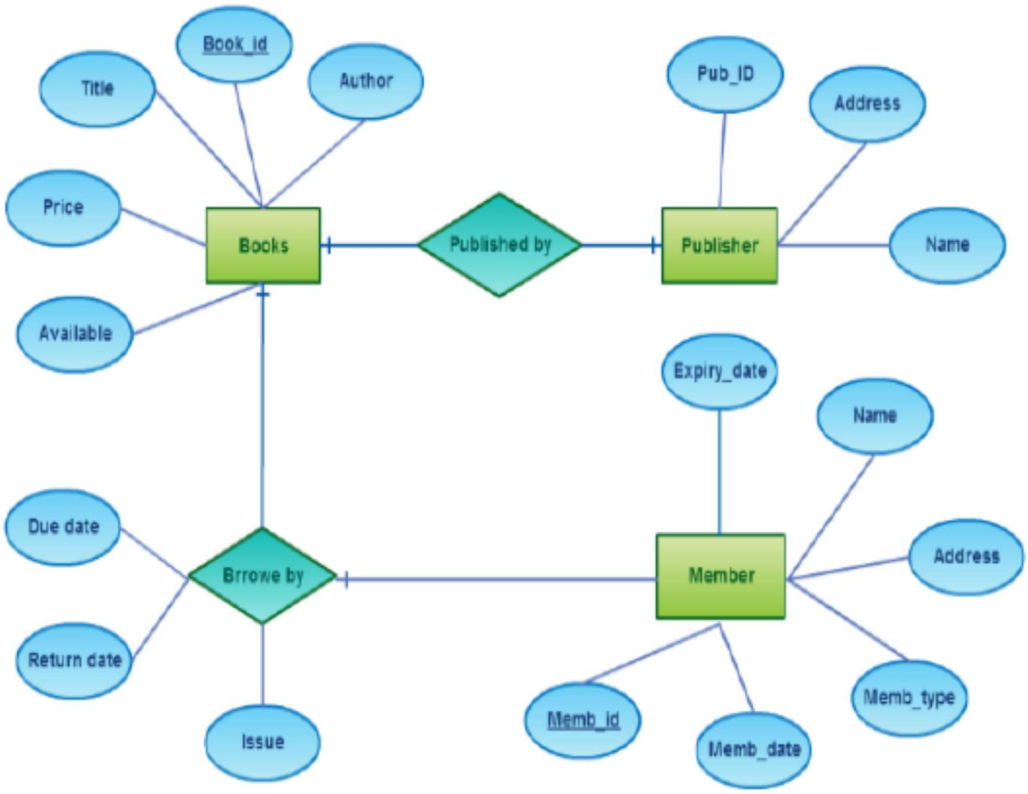
ER-DIAGRAM FOR HOSPITAL MANAGEMENT SYSTEM (SAMPLE)



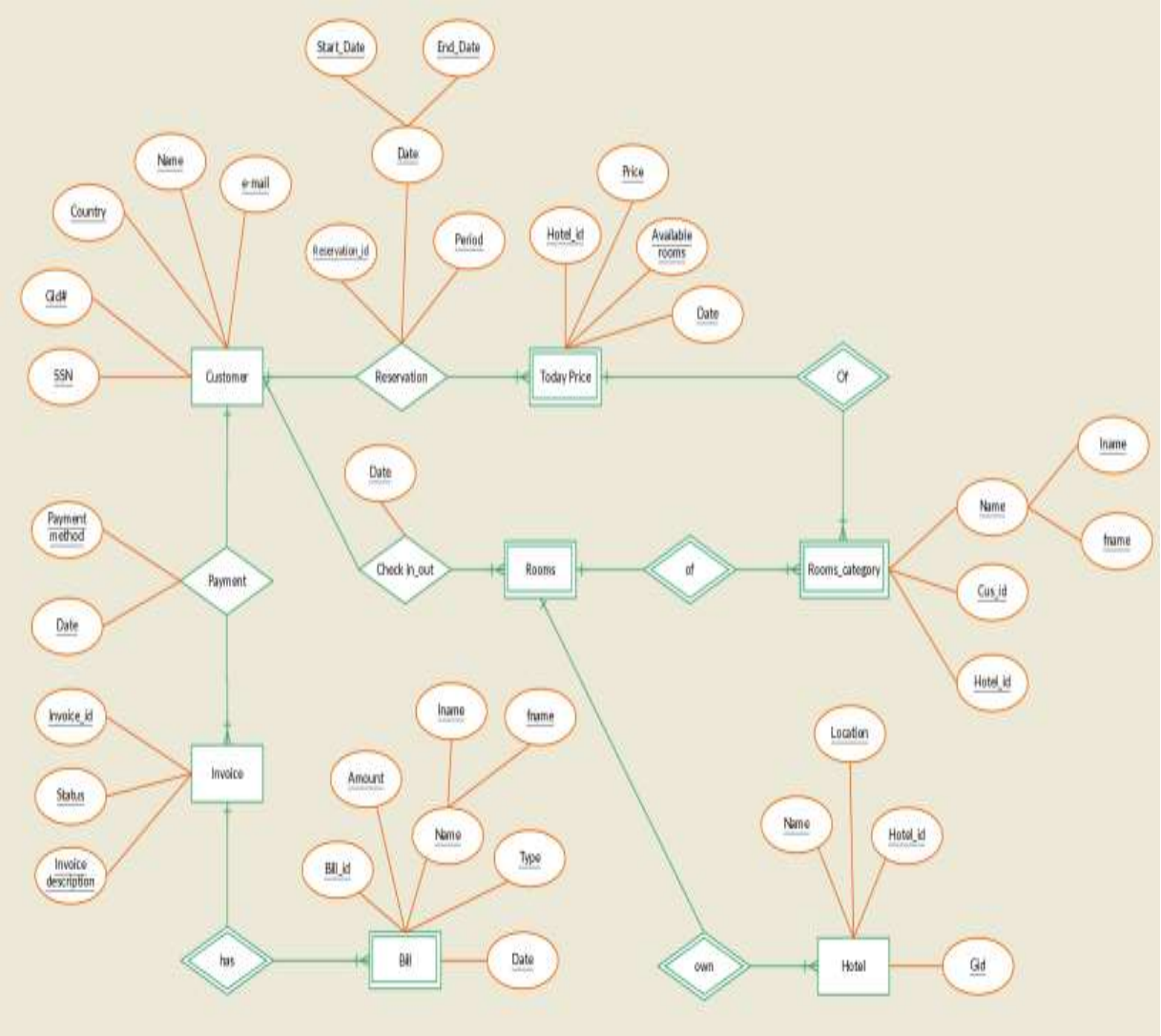
ER-DIAGRAM FOR EXAMINATION SYSTEM



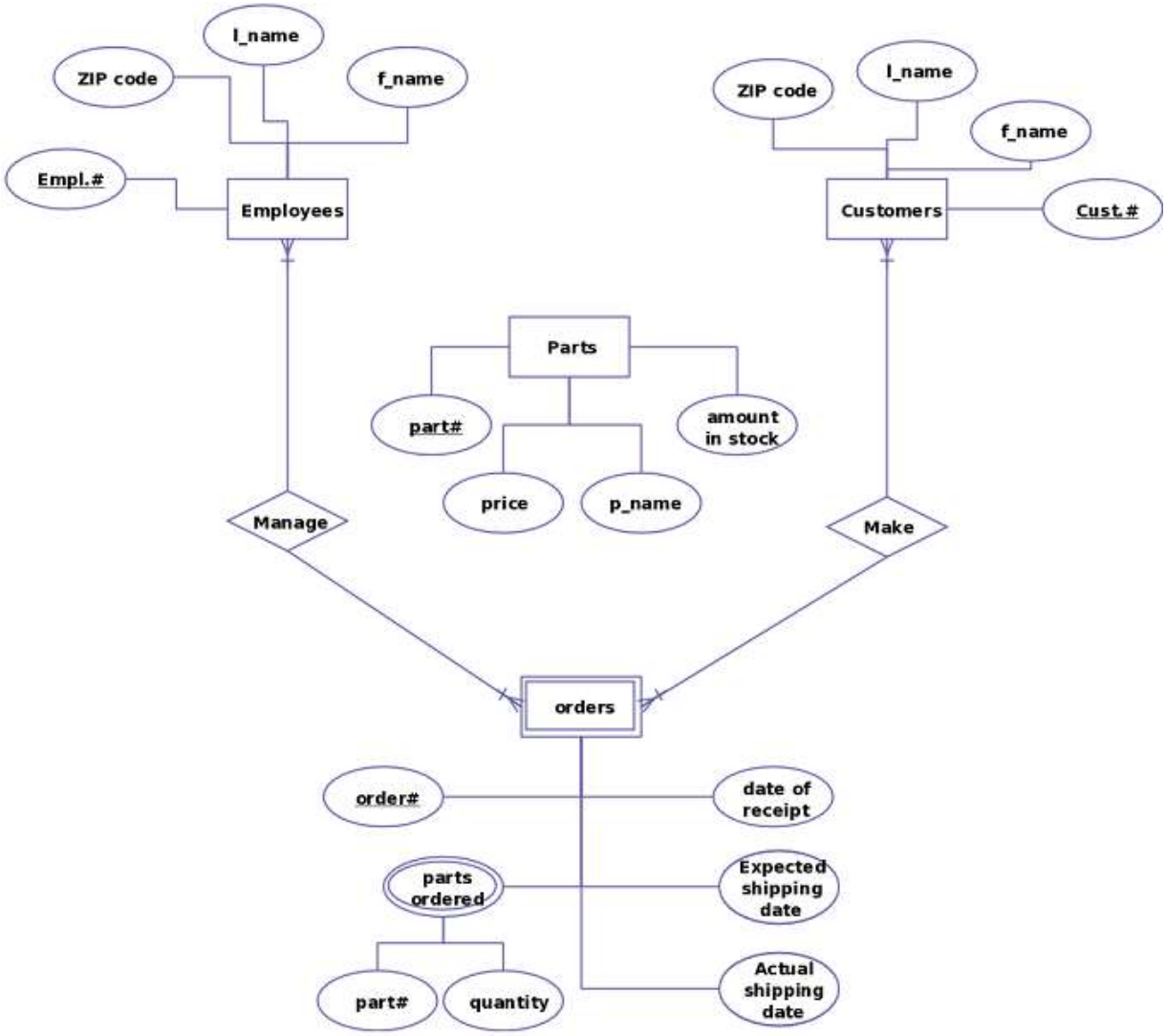
ER-DIAGRAM FOR LIBRARY SYSTEM



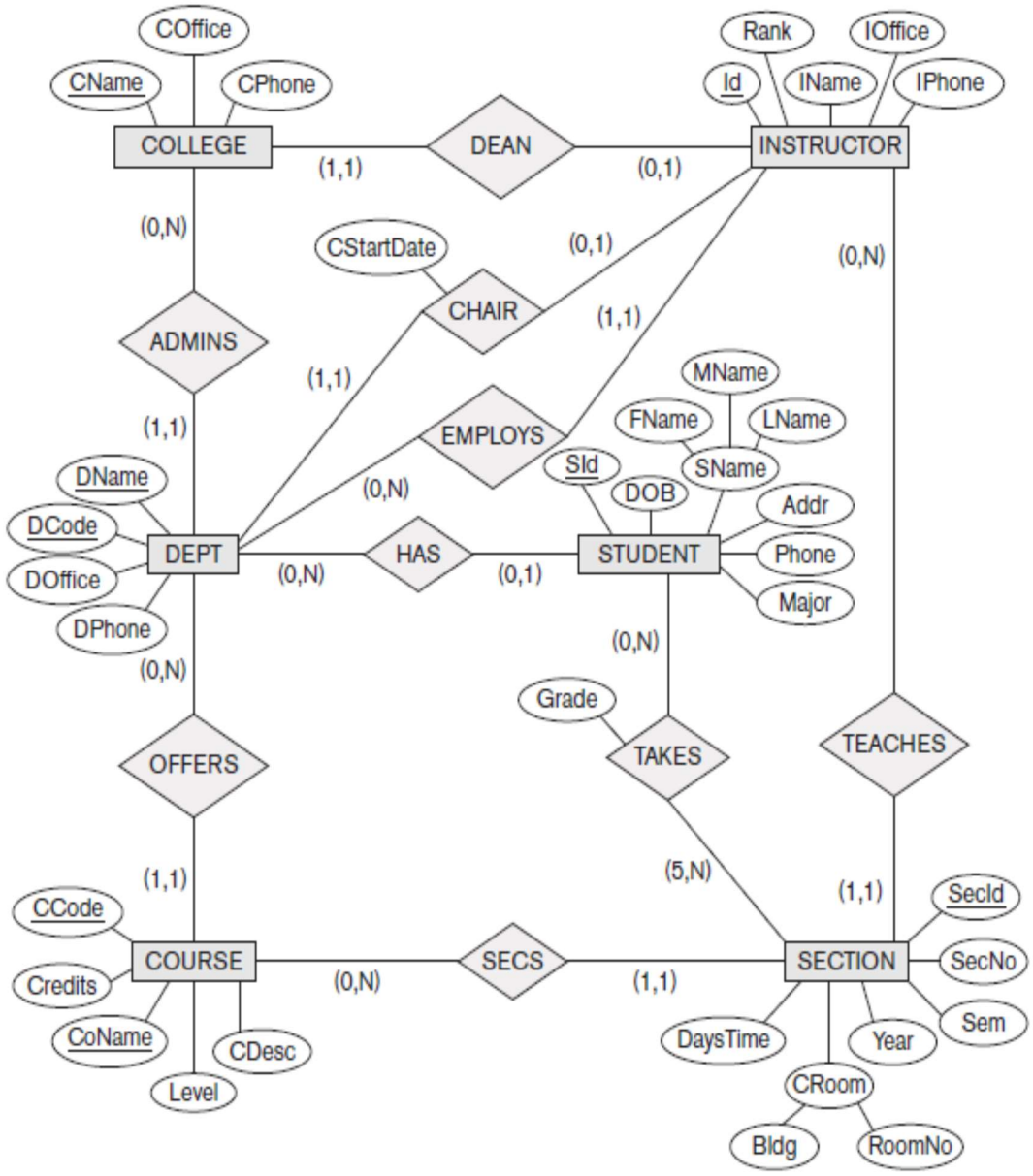
ER-DIAGRAM FOR HOTEL MANAGEMENT



ER-DIAGRAM FOR MAILORDER SYSTEM(SAMPLE)



ER Diagram for UNIVERSITY Database



REVIEW QUESTIONS

1. What are the structural constraints on a relationship type? Explain with Examples? (05 marks)
2. What is a weak entity type? Explain the role of partial key in design of weak entity type? (05 marks)
3. Define/Explain the following terms with examples:
   - Recursive relationship.
   - Weak entity type.
   - Atomic attributes.
   - Participation constraint.
   - Cardinality ratio.
   - Ternary relationship.
   - Entity.
   - Strong entity.
   - Composite attribute.
   - One to many relationship.
   - Weak entity set. (2x11 marks)
4. Define and explain partial key with example. (04 marks)
5. What is meant by recursive relationship? Explain the importance of role Names in recursive relationship with example. (08 marks)
6. What is the concept of weak entity type used in data modeling? Define owner entity type and identifying relationship type. (06 marks)
7. Describe how to map the following in ER-model to schema
   i) Strong entity
   ii)One to one relationship (10marks)
8. What is cardinality ratio and its types? (04marks)